

Manual de Usuario MongoDB



Docker & Visual Code

Guía completa para principiantes: instalación, configuración y uso.



mongoDB®

INDICE

Sección	Contenido	Página aprox.
Parte 0	Introducción	2
Parte 1	Objetivo del manual	3
Parte 2	Requisitos previos	3
Parte 3	Instalación de MongoDB	4
Parte 4	Instalación del entorno	4
Parte 5	Descarga de la imagen de MongoDB	5
Parte 6	Descarga y ejecución del contenedor	6
Parte 7	Persistencia de datos, contenedor descargado	7
Parte 8	Creación del Contenedor de MongoDB en Docker	8
Parte 9	Comando para crear el contenedor de MongoDB	9
Parte 10	Administración del contenedor	10
Parte 11	Visual Studio Code y la conexión con MongoDB	11
Parte 12	Visual Studio Code y Extensiones	12
Parte 13	MongoDB for VS Code	13
Parte 14	Instalar Editor de confianza	14
Parte 15	Connect. (Conexión de Visual a Contenedor Docker)	15
Parte 16	Montar Base de Datos Academia	16
Parte 17	Darle identidad al archivo (Formato .js)	17
Parte 18	Nombrar archivo a formato ".js" (gestión academia)	18
Parte 19	Bloque o "Limpieza de bases de datos" Academia	19
Parte 20	Bloque 1 "Creación de Colecciones" Usuarios en Academia	20
Parte 20	Bloque 2 "Inserción de un único usuario" "Alex López"	20
Parte 21	Bloque 3 "Comando insertOne" Colección Usuarios	21
Parte 22	Bloque 4 "Inserción Cursos Colección" Con insertMany	22
Parte 22	Bloque 5 "Consulta todos Usuarios" Almacenados Colección	22
Parte 23	Bloque 6 "Consulta Usuarios" Con interés Kotlin	23
Parte 24	Bloque 7 "Consulta Usuarios" Con interés Java, JS - \$in	24
Parte 24	Bloque 8 "Actualización Precio Curso Kotlin" Con updateOne	24
Parte 25	Bloque 9 "Comprobación Precio Actualizado" Curso Kotlin	25
Parte 26	Consulta Adicional "Buscar Precios" Superior 20 €	26
Parte 27	Operación Extra "Eliminación Usuario" Con deleteOne	27

INTRODUCCIÓN

DESCRIPCIÓN GENERAL

MongoDB

Es una base de datos NoSQL orientada a documentos. En lugar de usar tablas, guarda la información en documentos JSON con un esquema flexible, lo que permite modificar su estructura sin problemas. Se adapta fácilmente al crecimiento de los datos gracias a su escalabilidad horizontal.

Instalación de MongoDB

Descarga el instalador desde el sitio [oficial](#), selecciona la versión Community Server compatible con Windows (Windows Server 2008 R2 64 bits o superior con soporte SSL). Luego, ejecuta el instalador y sigue las instrucciones hasta finalizar la instalación.

Docker

Es una herramienta que permite ejecutar aplicaciones dentro de contenedores, entornos aislados que incluyen todo lo que una aplicación necesita para funcionar (código, dependencias y configuraciones). Facilita el desarrollo, las pruebas y el despliegue de proyectos al eliminar problemas de compatibilidad entre distintos equipos o servidores.

Instalación de Docker

Descarga Docker Desktop para Windows, ejecuta el instalador, completa los pasos de configuración y abre la aplicación. Para comprobar su funcionamiento, usa la terminal y ejecuta:

```
docker -version.
```

Si muestra la versión instalada, Docker está listo para usarse.

Visual Studio Code (VS Code)

Es un editor de código gratuito desarrollado por Microsoft. Compatible con múltiples lenguajes y sistemas operativos, ofrece una interfaz ligera y extensible mediante complementos. Es ideal para programar y trabajar con herramientas como MongoDB y Docker, ya que permite integrar terminales, extensiones de bases de datos y control de versiones en un solo entorno.

Instalación de Visual Studio Code

Entra en la página [oficial](#) code.visualstudio.com, descarga la versión para Windows y ejecuta el instalador. Una vez instalado, puedes agregar extensiones desde la pestaña Extensions (por ejemplo, para Docker, MongoDB o GitHub).

Objetivo del Manual

El objetivo de este manual es guiar al usuario en la puesta en marcha de MongoDB dentro de un contenedor Docker, proporcionando instrucciones claras para:

- Instalar el entorno necesario.
- Descargar y ejecutar la imagen oficial de MongoDB.
- Configurar almacenamiento persistente.
- Conectarse al servicio desde consola o herramientas externas.
- Realizar operaciones básicas de administración.
- Aplicar medidas mínimas de seguridad y mantenimiento.

Requisitos previos

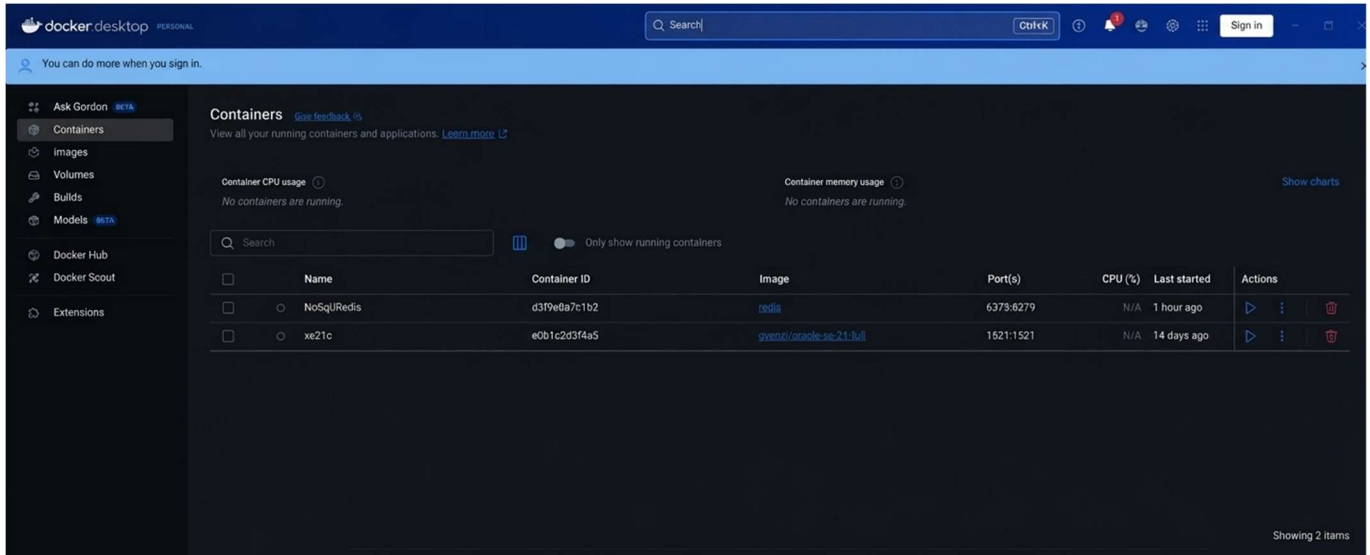
Antes de comenzar, el usuario debe disponer de lo siguiente:

- Un equipo con Windows, Linux o macOS.
- Docker instalado y funcionando correctamente.
- Acceso a terminal o consola del sistema.
- Conocimientos básicos sobre uso de línea de comandos.
- Permisos suficientes para crear y administrar contenedores.

De forma recomendable, también se sugiere contar con Docker Desktop en entornos Windows o macOS, o Docker Engine en Linux.

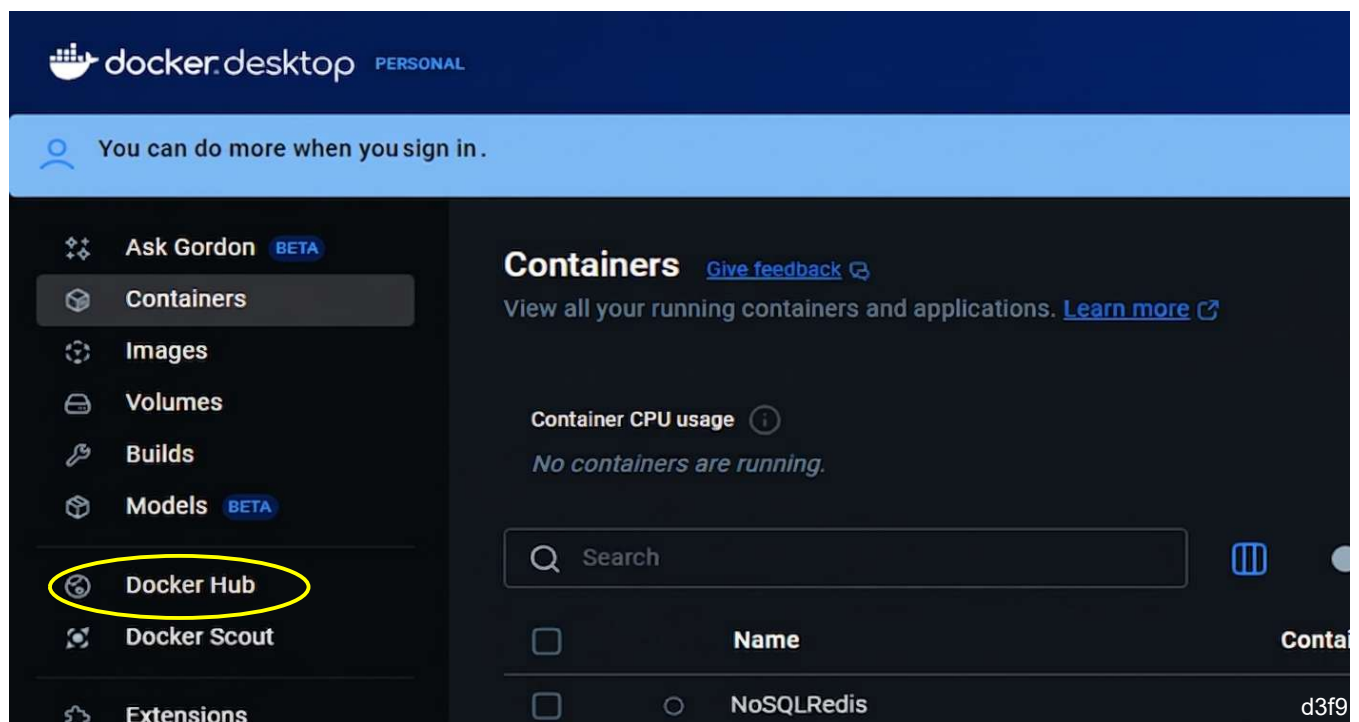
Instalación: de MongoDB usando Docker. Lo que necesitas es la versión Community Server, que es la gratuita y suficiente para trabajar.

Primero, abre la aplicación Docker en tu ordenador para poder empezar con la instalación.



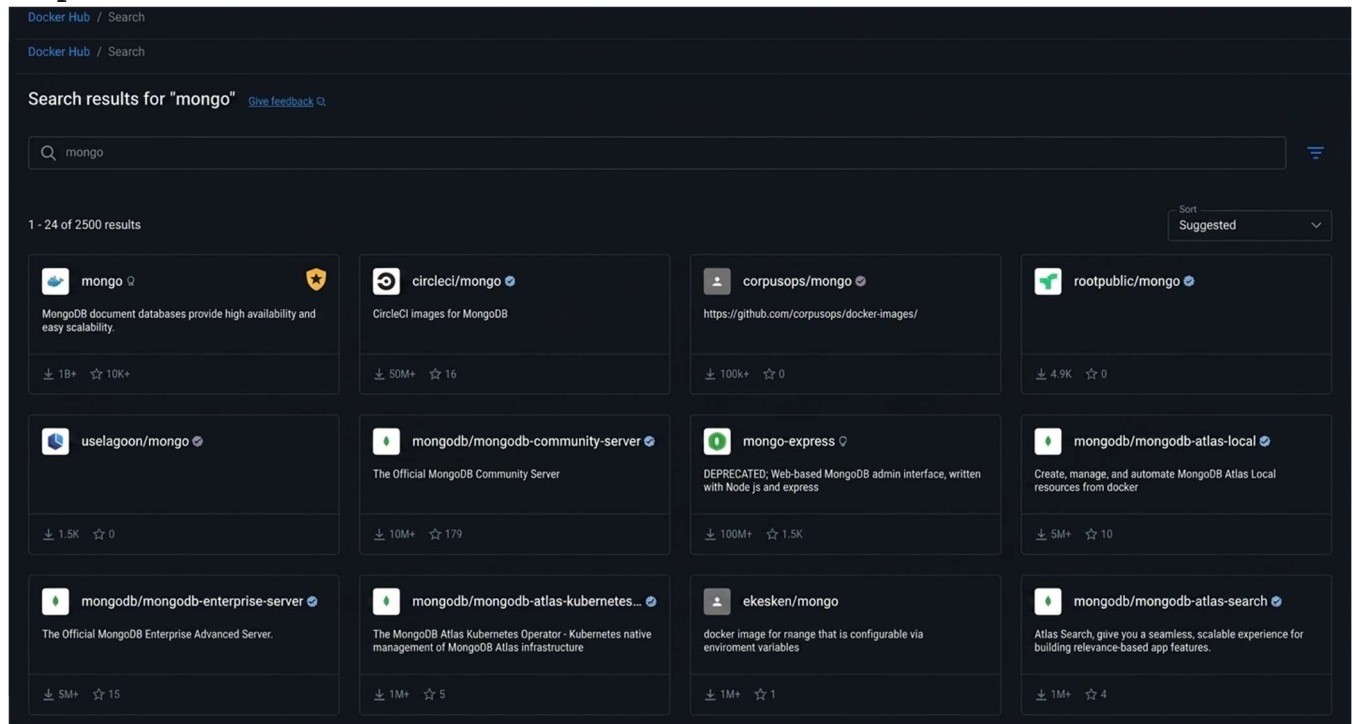
Ahora toca descargar la imagen de MongoDB desde Docker Hub.

Dentro de la aplicación Docker, ve a la sección Docker Hub, es ahí donde se encuentran todas las imágenes disponibles para usar.

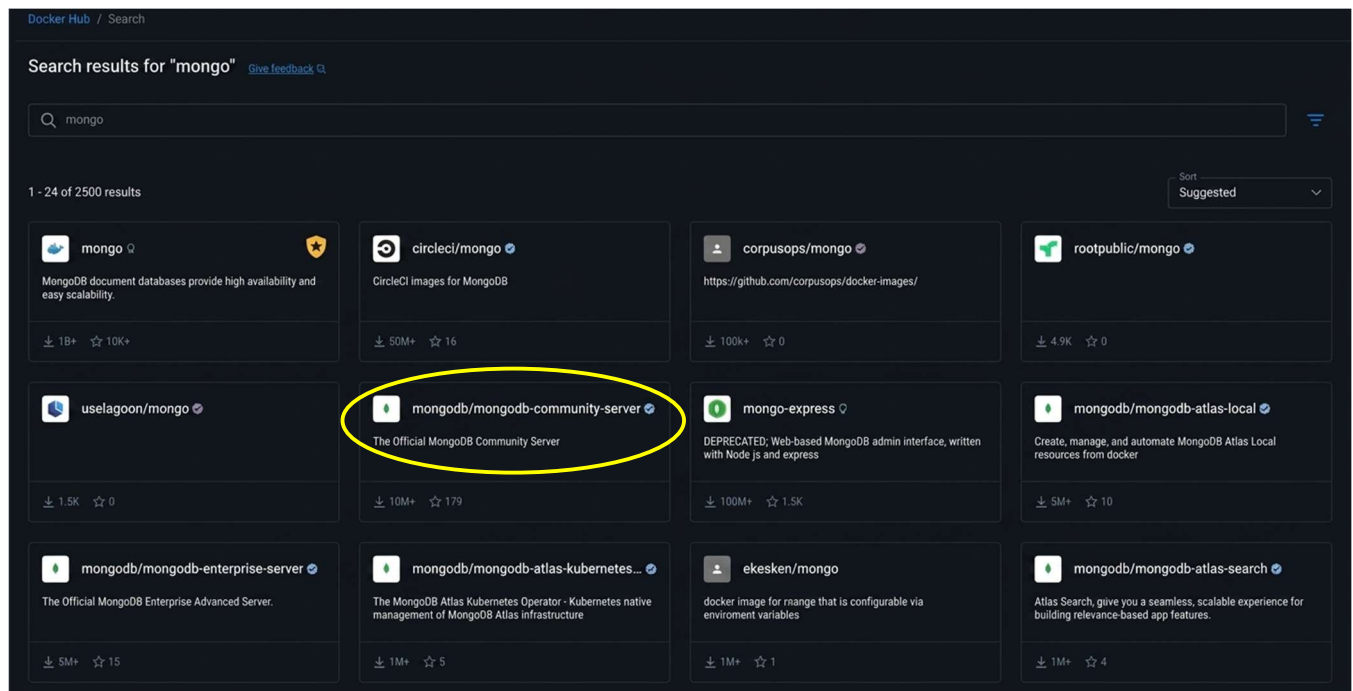


Una vez estés dentro de Docker Hub, escribe “mongo” en la barra de búsqueda para encontrar la imagen oficial de MongoDB.

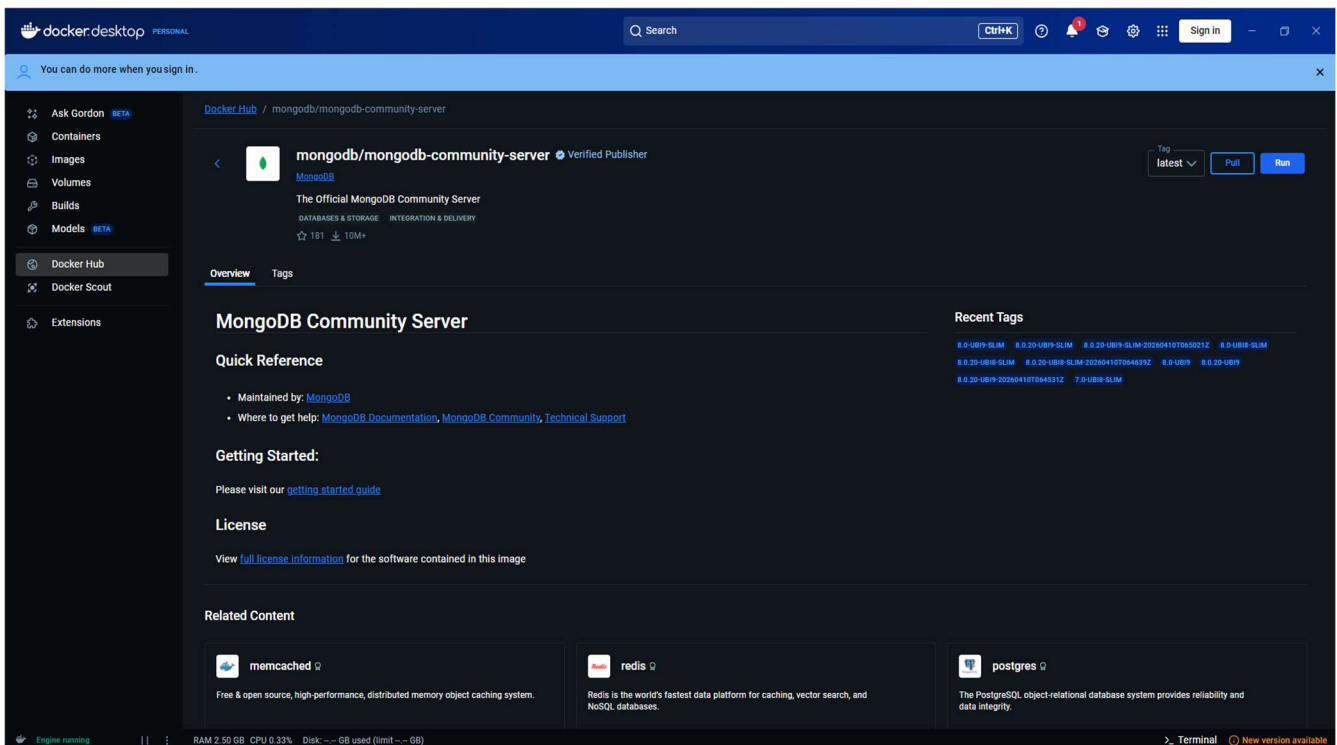
Luego haz clic en el botón Search y verás una lista con las distintas versiones disponibles.



Después, selecciona la opción “Community Server”, que es la versión gratuita y más utilizada de MongoDB para proyectos de desarrollo o pruebas.

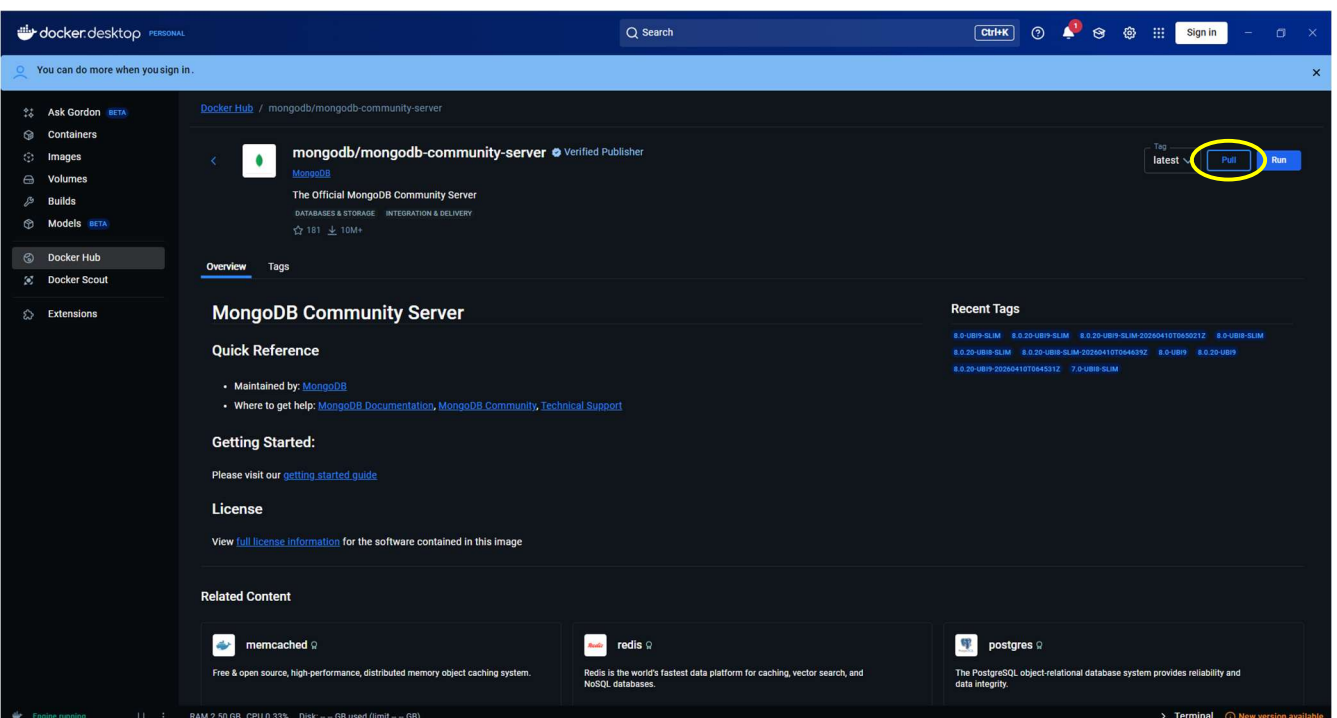


Después de hacer clic en la versión **Community Server**, se te abrirá una nueva pantalla con los detalles de esa imagen de MongoDB, preparada para que puedas revisarla o descargarla.

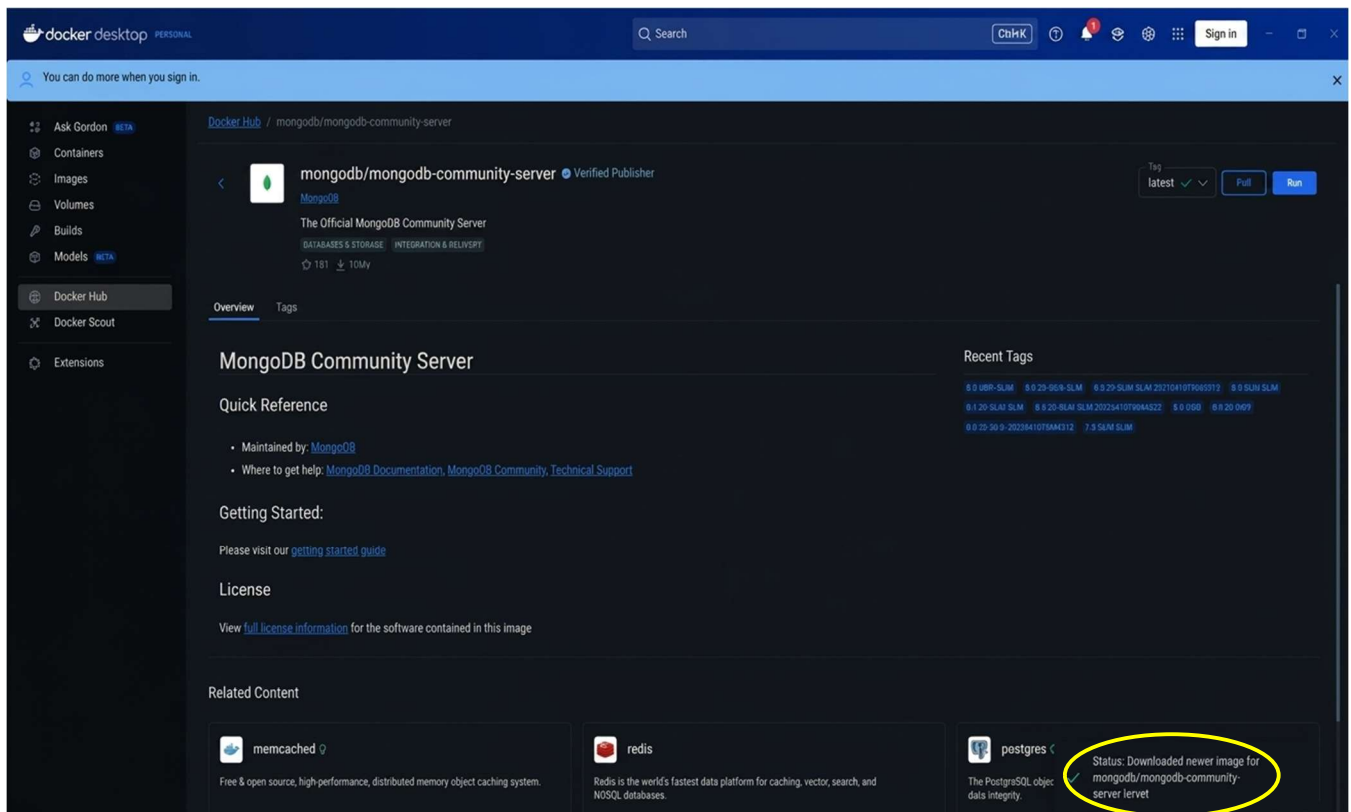


Antes de descargar nada, fíjate en el campo **TAG** y asegúrate de que tengas seleccionada la última versión disponible de la imagen de MongoDB Community Server.

Cuando veas que el tag es el correcto, haz clic en el botón “Pull” para empezar a descargar la imagen en tu Docker.



Cuando te aparezca esa notificación emergente, quiere decir que el **pull** ha terminado y que la imagen se ha descargado correctamente en tu Docker, lista para usarse en un contenedor.

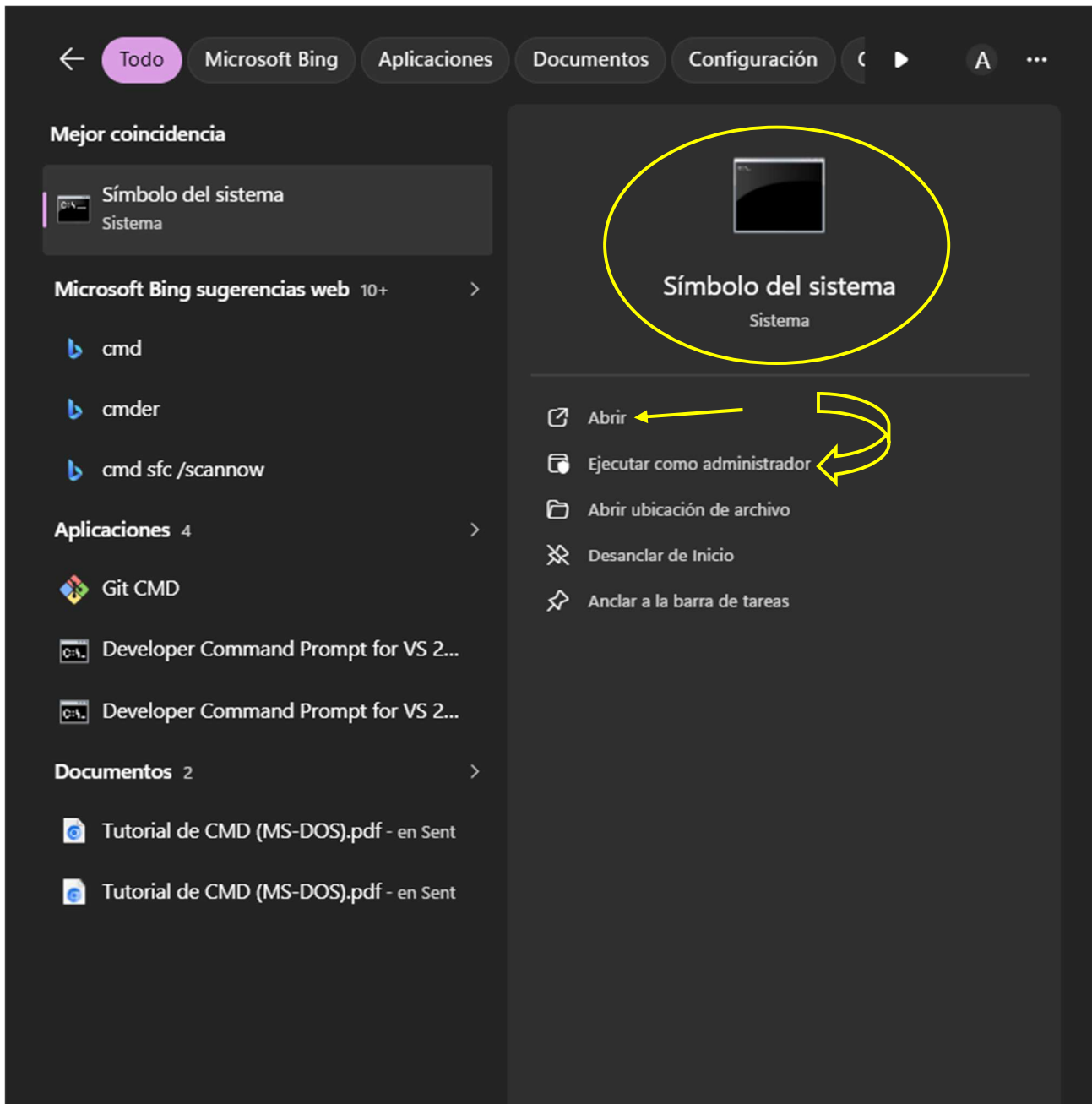


Pequeño recordatorio: deja la aplicación de Docker abierta y en ejecución, no la cierres, porque la vamos a seguir usando en los siguientes pasos para crear y gestionar el contenedor de MongoDB.

Ahora vamos a crear el contenedor de MongoDB en Docker: con una configuración básica para poder trabajar cómodamente.

- Pon como **nombre del contenedor**: mongo.
- Usa el **puerto 27017**, que es el puerto por defecto en el que escucha MongoDB.
- Configura el **usuario administrador** usando la variable de entorno **MONGO_INITDB_ROOT_USERNAME** con el valor admin.
- Asigna una **contraseña** al administrador mediante **MONGO_INITDB_ROOT_PASSWORD**, eligiendo una clave que recuerdes o tengas apuntada en un lugar seguro.

Para crear el contenedor vamos a usar comandos, así que el siguiente paso es abrir el “**Símbolo del sistema**” de Windows (la clásica ventana negra donde puedes escribir instrucciones como docker run).



Una vez tengas abierto el **Símbolo del sistema**, el siguiente paso es escribir y ejecutar el comando que va a crear el contenedor de MongoDB con la configuración que hemos definido antes.



En la terminal, pega y ejecuta el comando que te dejo aquí preparado para crear el contenedor con MongoDB y el usuario admin:

```
docker run -d --name mongo -p 27017:27017 -e
MONGO_INITDB_ROOT_USERNAME=admin -e
MONGO_INITDB_ROOT_PASSWORD=TU_CONTRASEÑA mongo
```

Copia la estructura del comando, pero cambia **TU_CONTRASEÑA** por una clave distinta a 1234, es decir, usa una contraseña propia y más segura para el administrador de MongoDB.

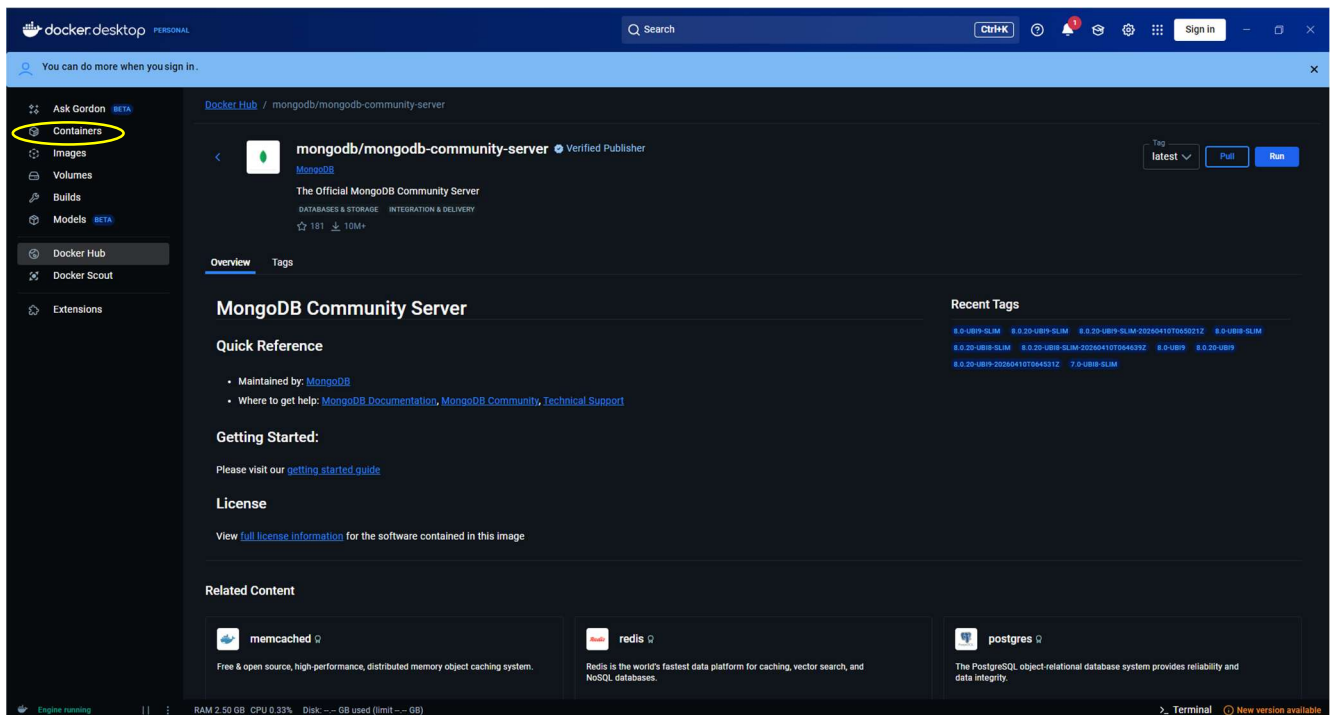
Después de lanzar ese comando en la terminal, debería salirte un resultado parecido a este, indicando que el contenedor se ha creado correctamente y ya está en marcha.



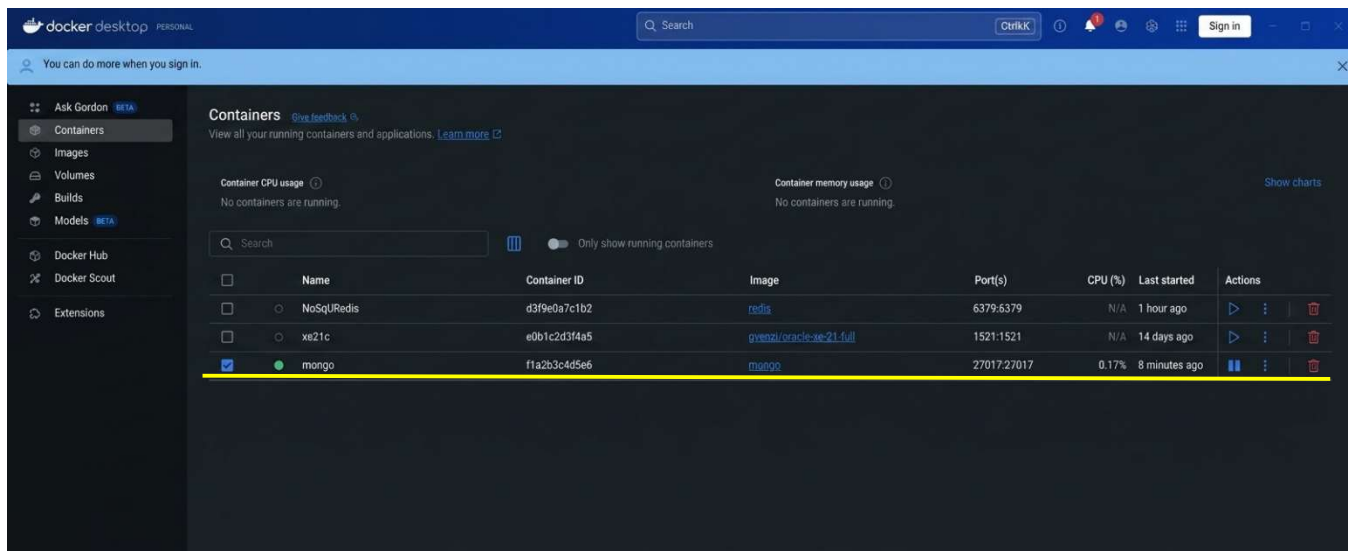
Cuando termine de ejecutarse el comando, en la terminal debería mostrarse algo parecido a esto, indicando que el proceso ha acabado y el contenedor se ha creado correctamente.



Cuando este paso haya terminado, vuelve a abrir **Docker Desktop** (si no lo tenías ya visible) y entra en la sección de **contenedores**, donde se listan todos los containers creados y en ejecución.



En esa lista de contenedores deberías ver uno llamado **“mongo”**, y su estado tiene que aparecer como en ejecución, indicando que el servicio de MongoDB está funcionando correctamente.

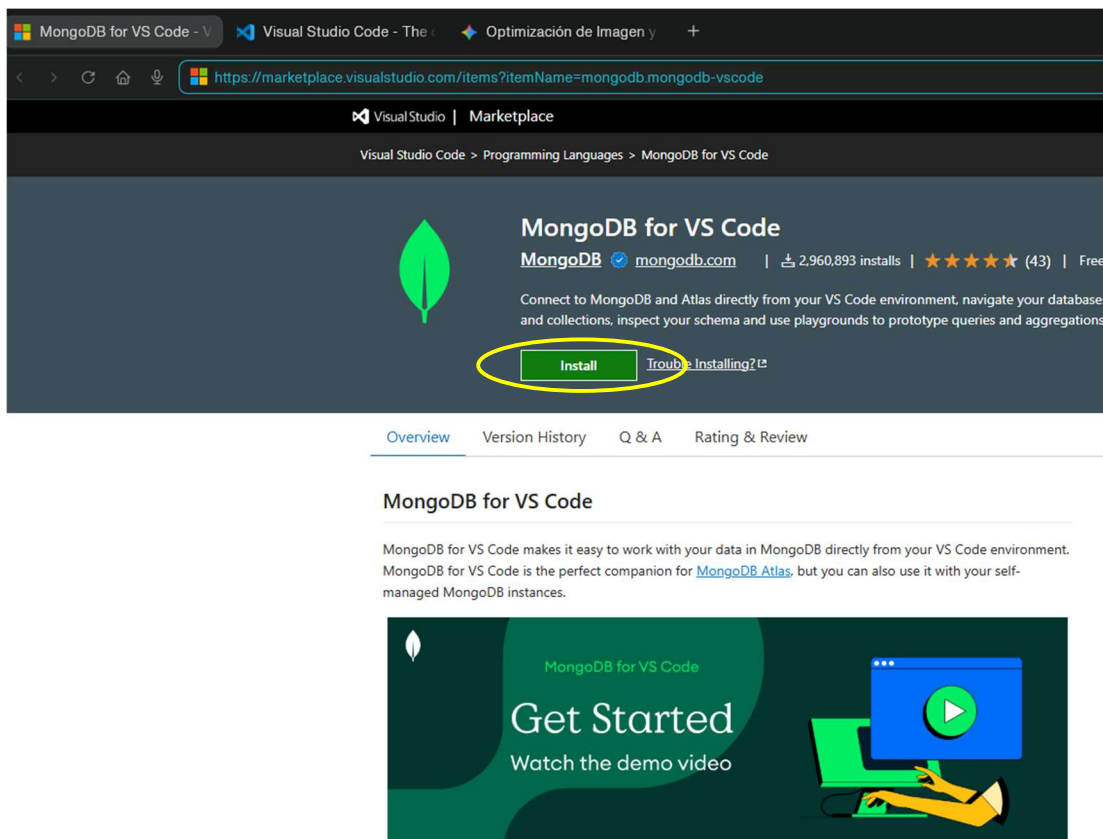


Si te fijas en los detalles de ese contenedor, verás que coincide con toda la configuración que hemos indicado antes en la terminal: nombre mongo, puerto 27017 y las variables de entorno del usuario administrador.

Ahora pasamos a la parte de **Visual Studio Code** y la conexión con **MongoDB**.

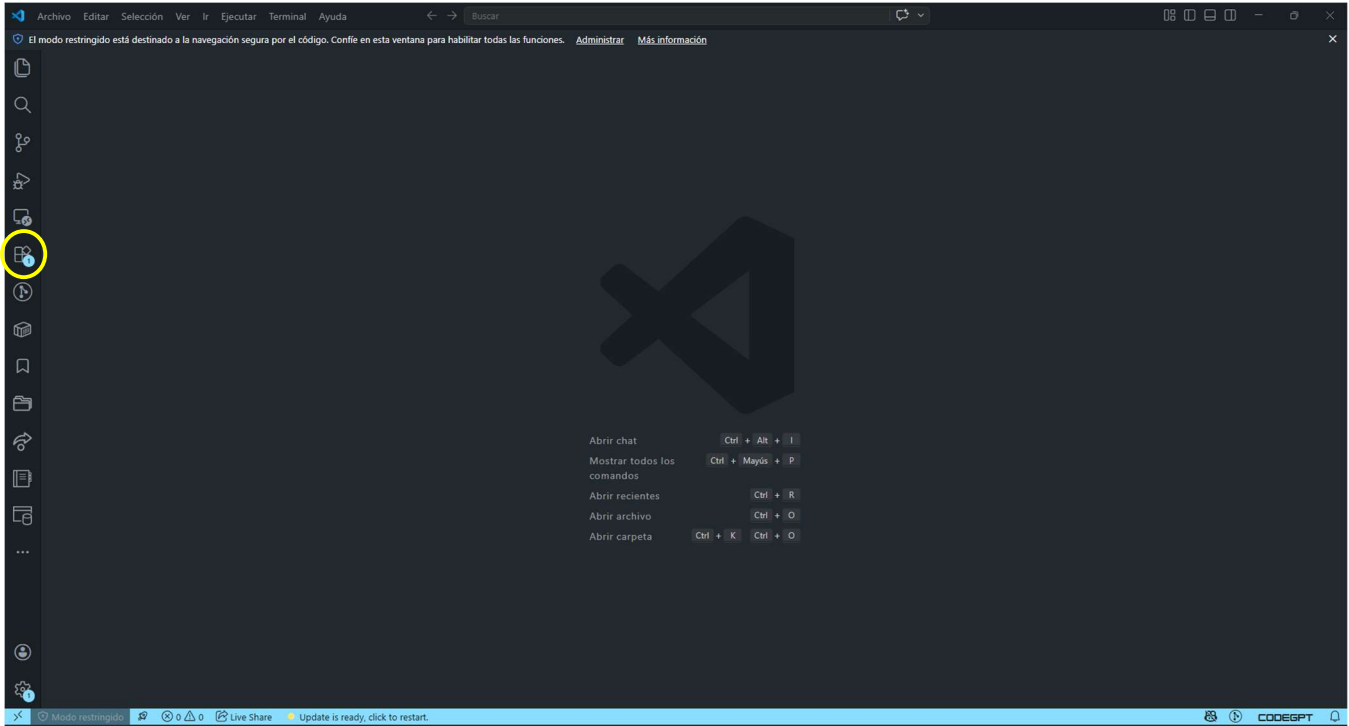
En este caso, como VS Code ya lo tenemos instalado en el equipo, nos centraremos directamente en configurarlo para que pueda conectarse en nuestra base de datos Mongo.

Extensión “**MongoDB for VS Code**”



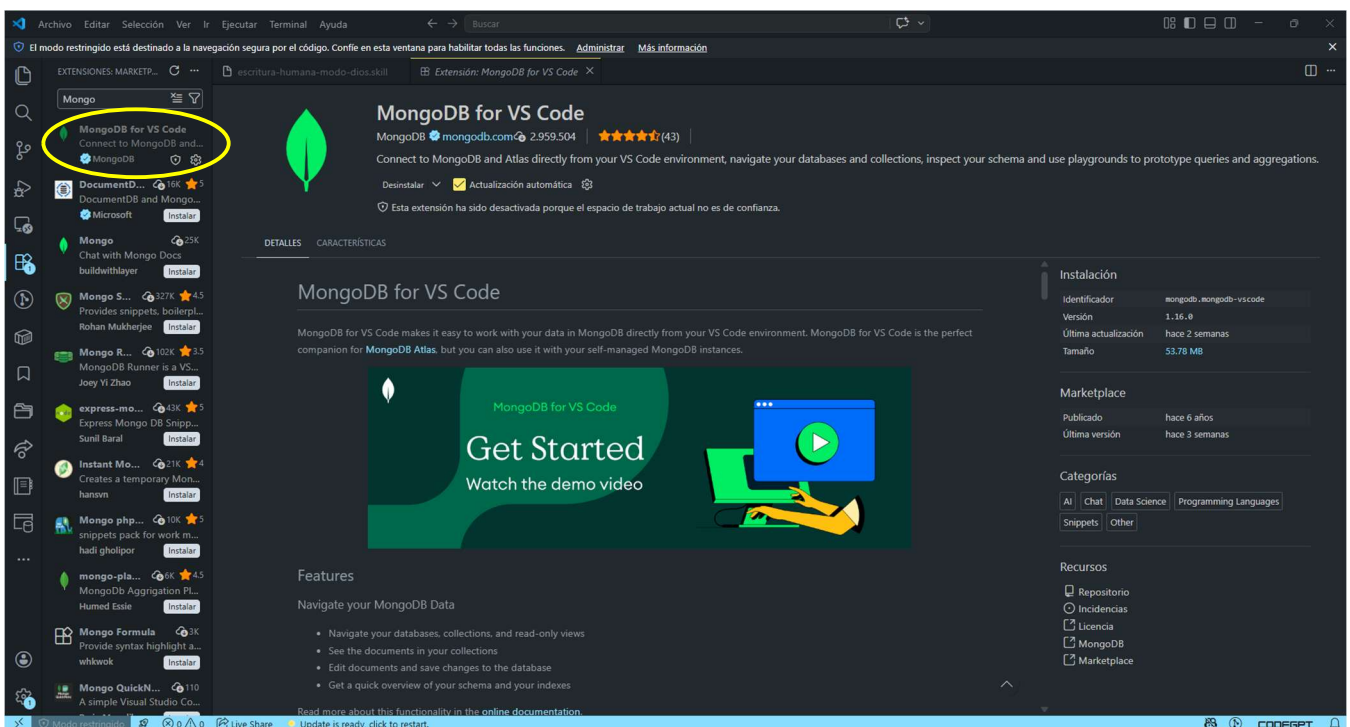
Para poder trabajar con la base de datos desde el editor, vamos a instalar el plugin de **MongoDB** en Visual Studio Code.

Abre **Visual Studio Code** y ve a la sección de **Extensiones** (el icono de los cuadraditos en la barra lateral izquierda).

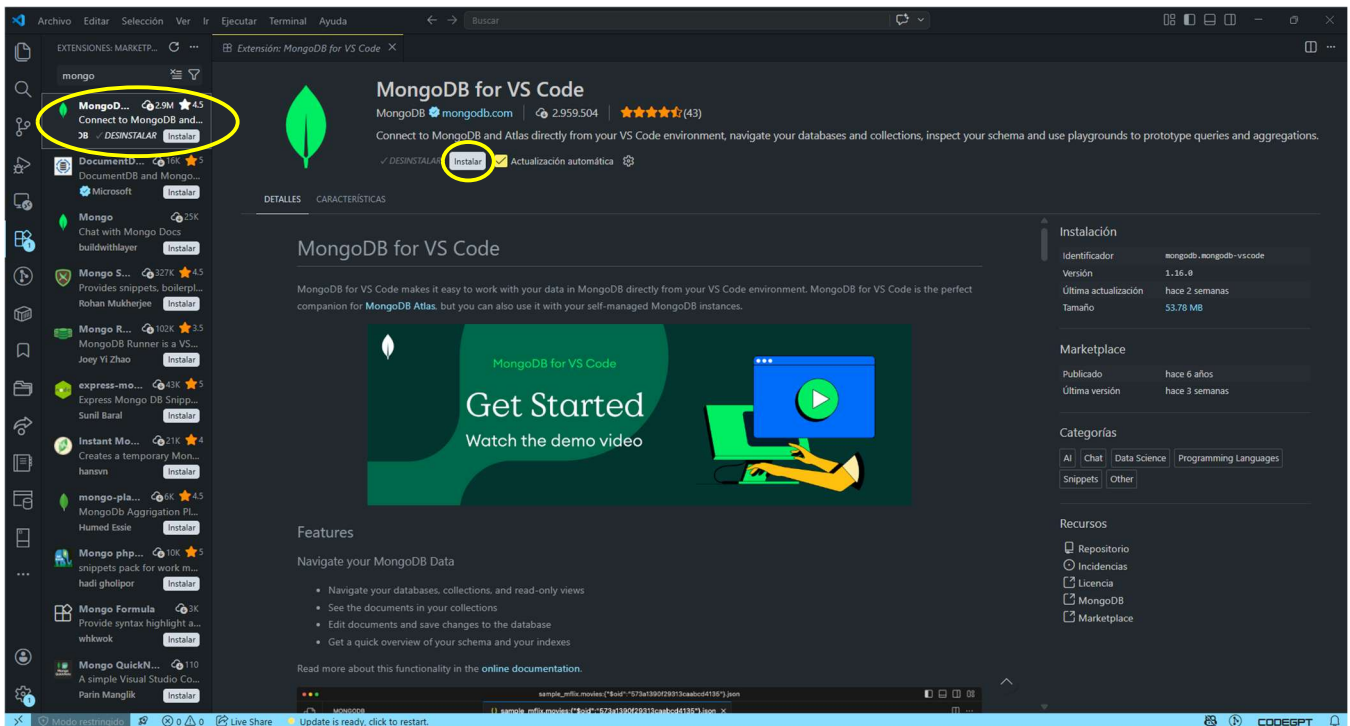


Dentro de la pestaña de **Extensiones**, escribe “mongo” en la barra de búsqueda para encontrar los complementos relacionados con MongoDB.

Entre los resultados, verás la extensión oficial llamada “**MongoDB for VS Code**”, que es la que vamos a instalar.

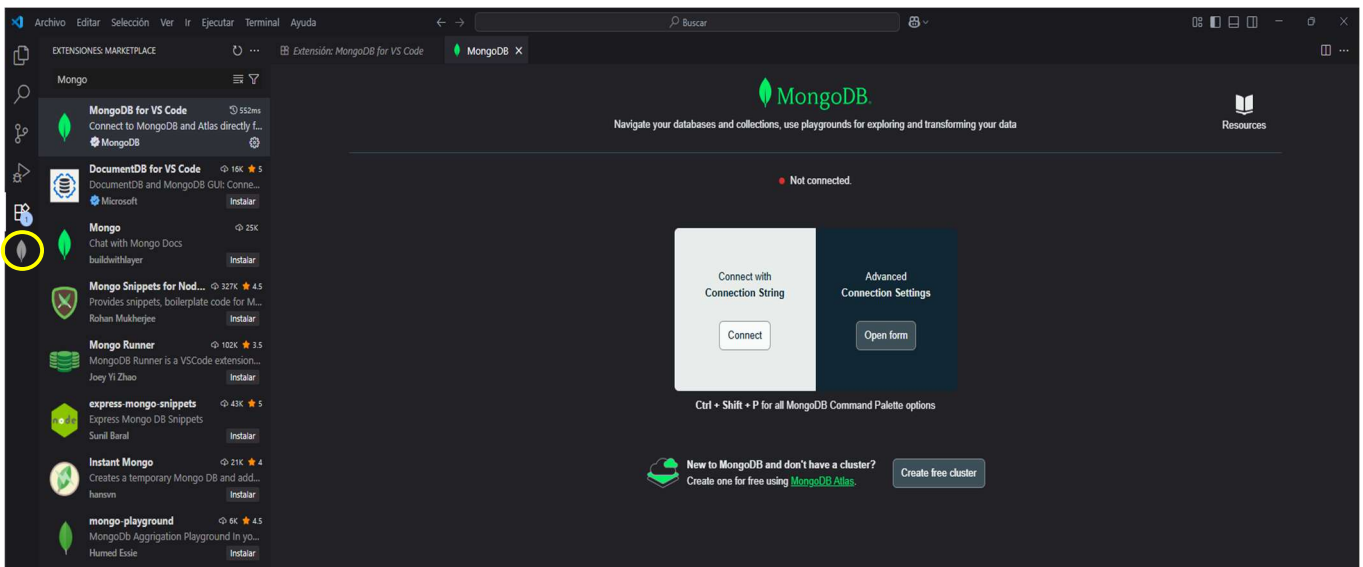


Cuando entres en la ficha de la extensión, se te abrirá una pantalla con la información y detalles de “MongoDB for VS Code”. Ahí simplemente haz clic en el botón **Instalar** para añadirla a tu Visual Studio Code.



Cuando la extensión se haya instalado bien, verás que aparece en la barra lateral izquierda un nuevo icono con la forma de una hoja verde, que corresponde a MongoDB.

Ese icono indica que la extensión de MongoDB para VS Code está activa y lista para usarse.



Ahora lo que tenemos que hacer ahora es súper sencillo. Básicamente, es elegir el **Editor de confianza** que preferimos y le damos a **instalar**.

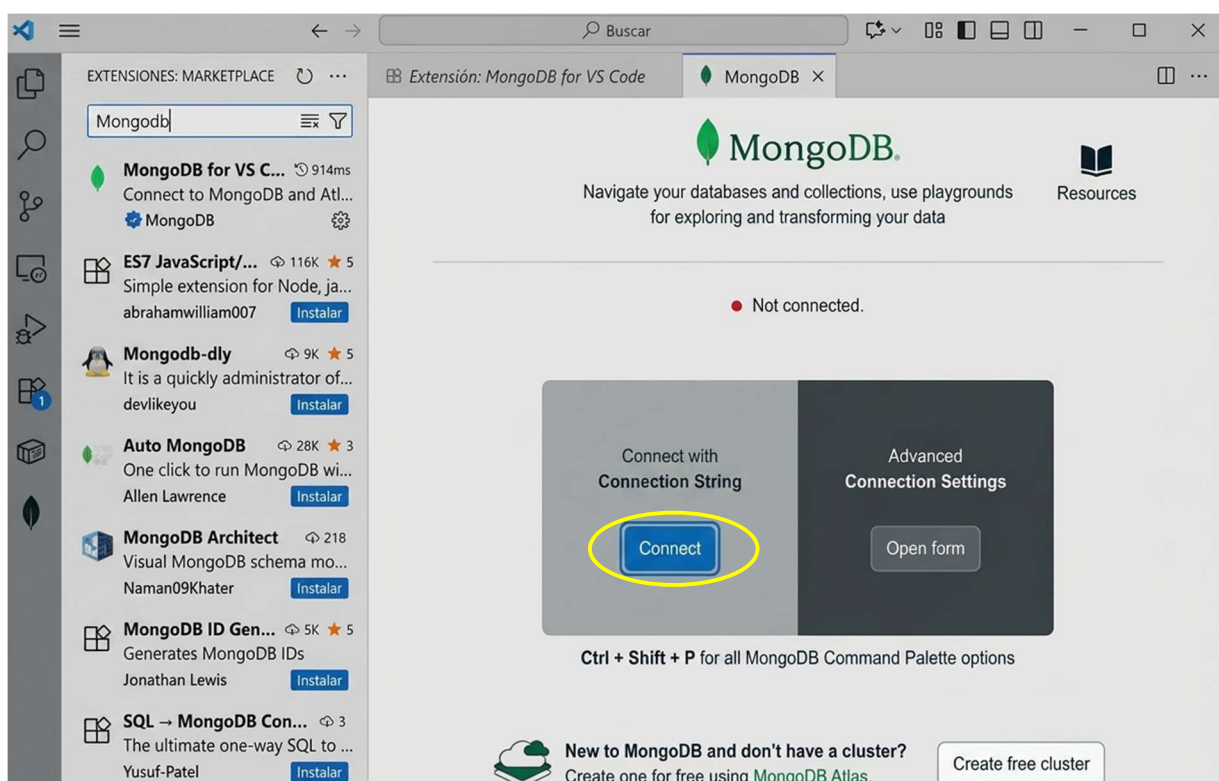
No tiene mucha pérdida, es solo marcar esa opción y seguir adelante con la instalación.



Aquí mucho cuidado que en este paso es el que suele dar más problema si no despistamos.

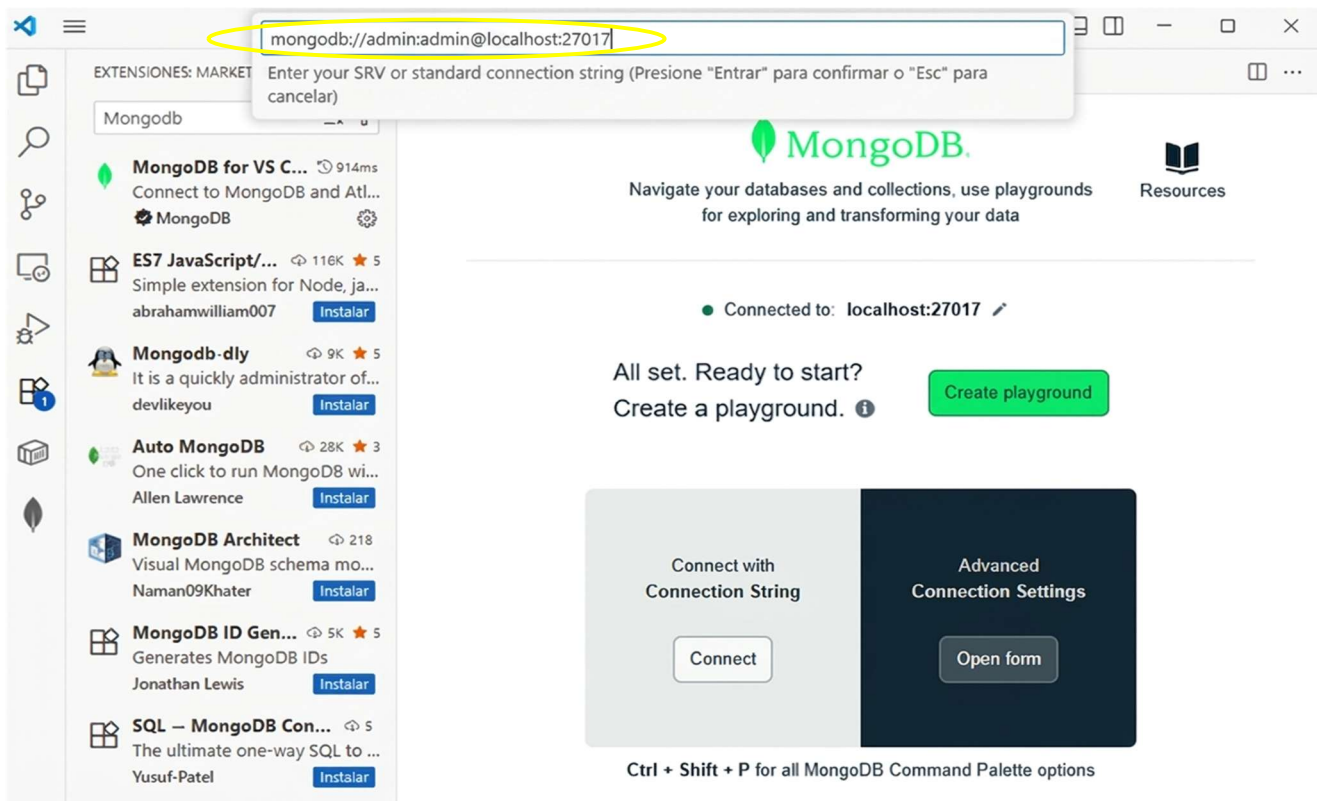
Se nos va a abrir la ventana de la extensión y ahí tenemos que darle a **Connect**.

Y esto es súper importante, tenemos que **asegurarnos de tener el contenedor abierto** antes de pulsar nada. Si no lo tenemos corriendo, nos va a dar un error o simplemente no va a conectar, así que demos un vistazo rápido al contenedor primero y luego ya hacemos clic. No tiene más, pero sí nos saltamos este paso no vas a poder seguir.



Cuando pulsemos en conectar, se abrirá un cuadro de búsqueda en el que tendremos que escribir lo siguiente:

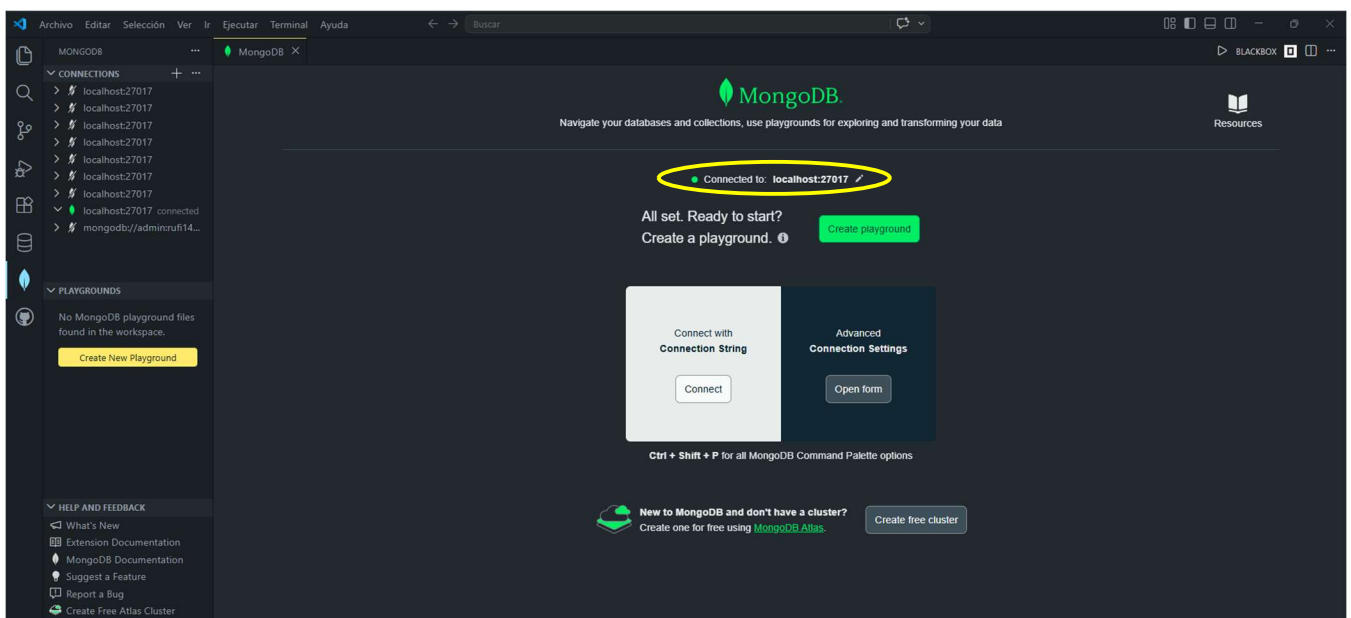
mongodb://admin:admin@localhost:27017.



Mira ahora, si todo va bien, deberíamos ver esto esta conexión que nos dice que hemos tenido éxito. Es la señal de que ya estamos dentro.

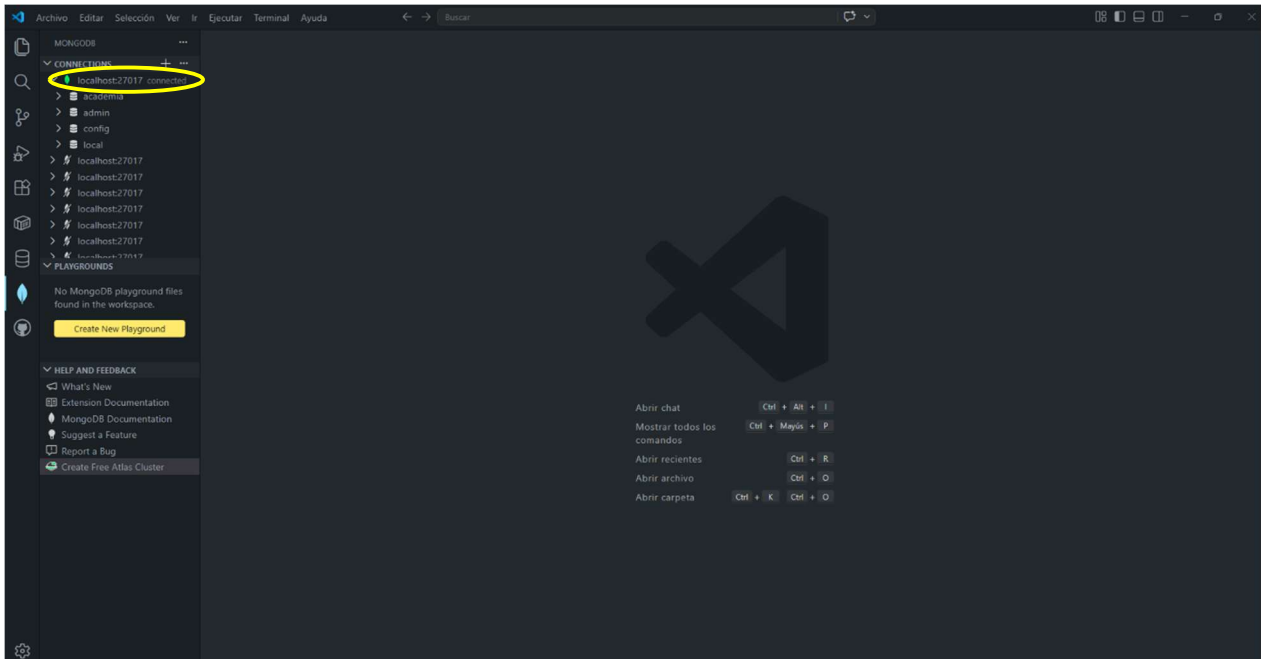
Realmente, en cuanto le damos a conectar y el sistema hace el "link" o conexión con el contenedor, nos salta este mensaje o aviso en pantalla. Si lo ves, quédate tranquilo porque significa que la conexión está activa y funcionando como debe ser.

Si no nos sale algo así...bueno, seguramente es que el contenedor se ha cerrado o ha pasado algo raro, pero vamos, que lo normal es que nos aparezca esto y ya lo tengamos confirmado.

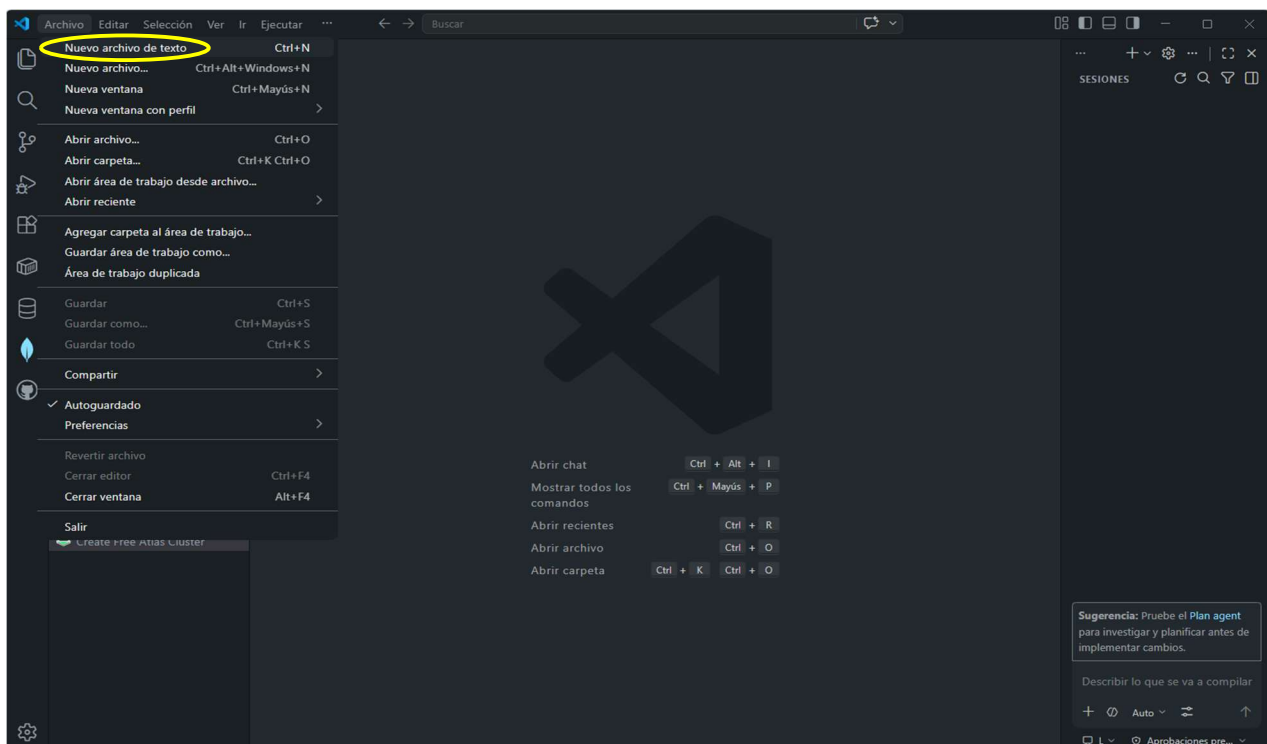


Lo siguiente es montar la base de datos **academia**. Para no complicarnos, vamos a generar un archivo llamado `gestión_academia.mongodb.js` donde meteremos los comandos necesarios. Dentro de este script, simplemente le diremos al sistema que se sitúe en la base de datos **academia** y, justo después, daremos la orden de crear las colecciones de **usuarios** y **cursos**."

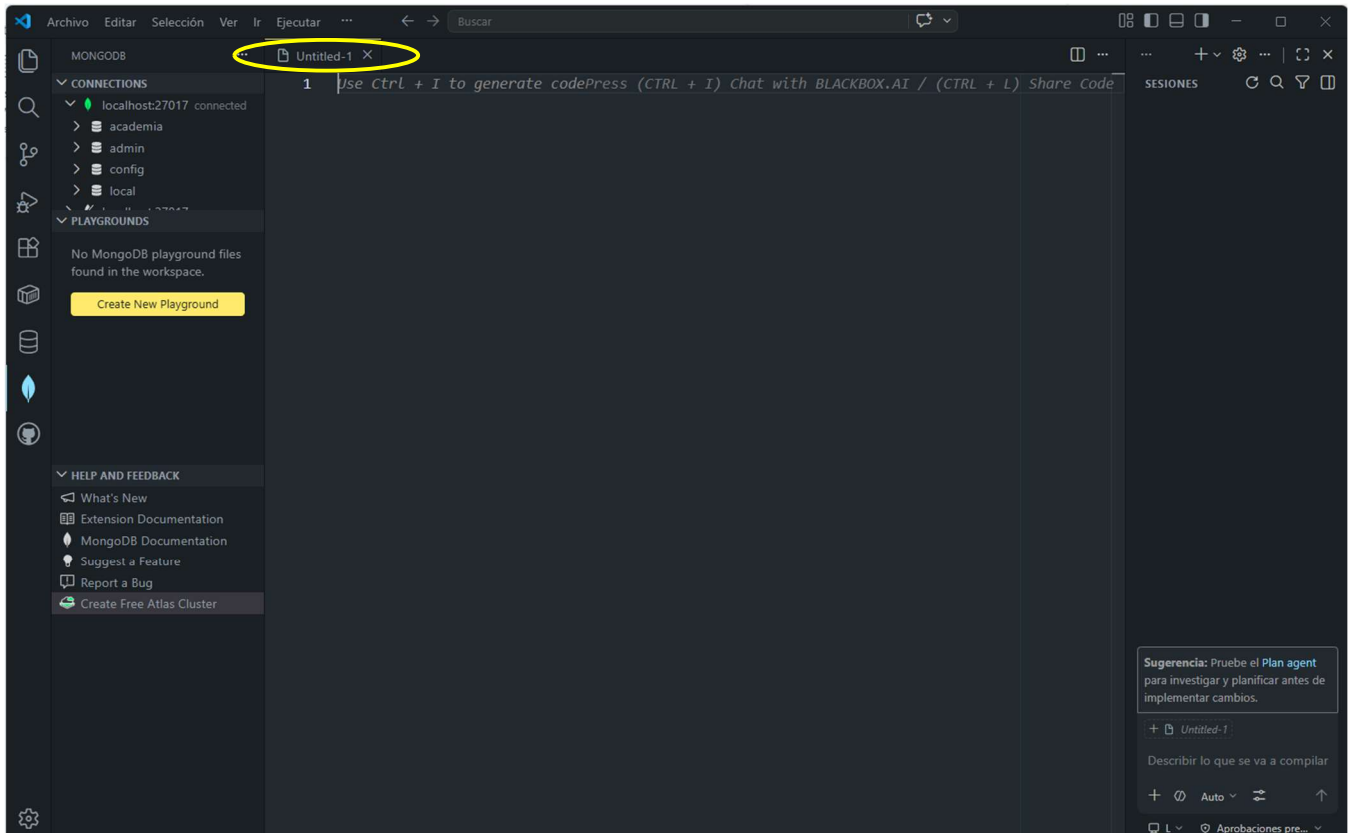
Ahora nos situarnos en el entorno de Visual Studio Code. Desde aquí es donde gestionaremos tanto los archivos de configuración como la conexión con el servidor que tenemos corriendo en Docker.



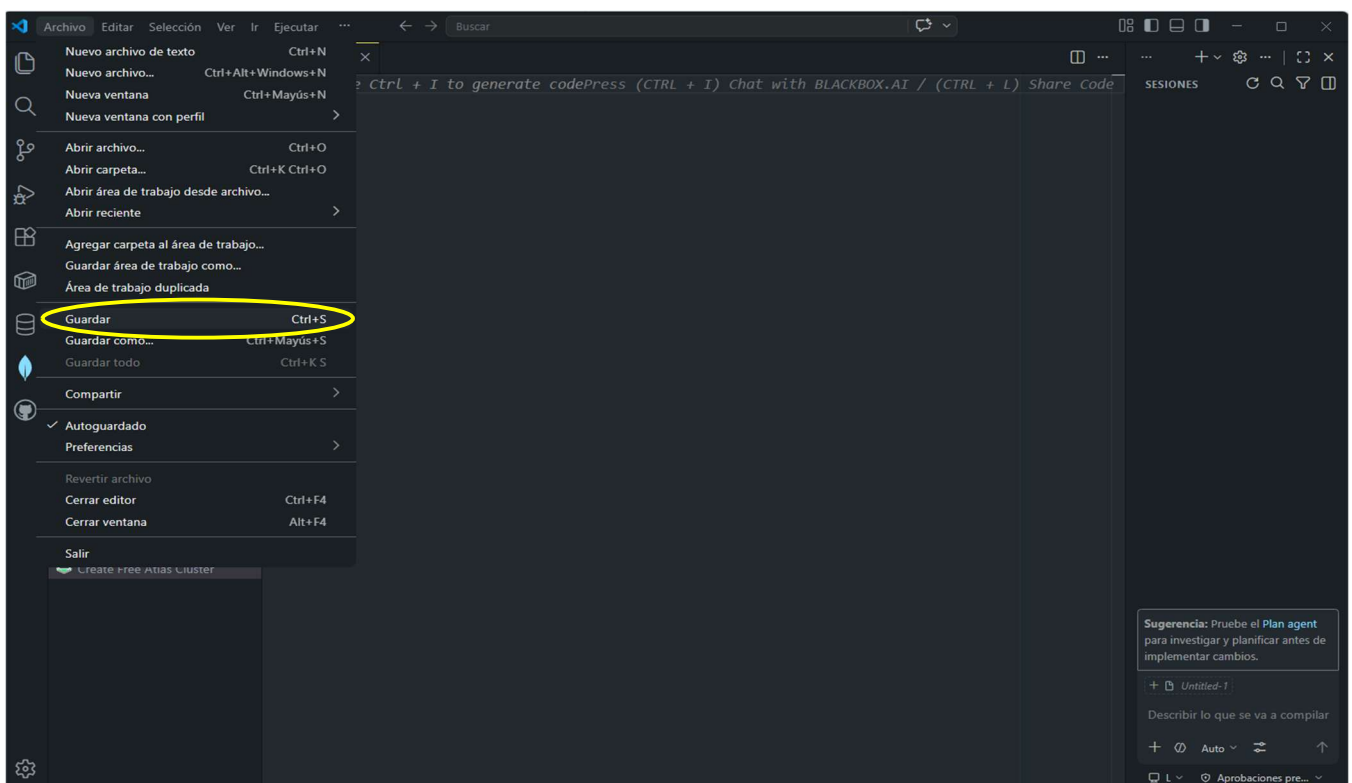
El siguiente paso es sencillo: simplemente dirígete a la parte superior izquierda de la pantalla, busca la opción **Archivo > Nuevo archivo de texto** y haz clic. Es el folio en blanco donde vamos a configurar nuestra base de datos.



Con la pestaña vacía delante, lo que toca ahora es **darle identidad al archivo**. Vamos a guardarlo con el nombre que definimos antes, para que Visual Studio Code sepa que lo que vamos a escribir dentro son comandos de MongoDB."



Una vez tenemos el archivo listo, no se nos puede olvidar guardarlo correctamente. Nos movemos a la esquina superior izquierda, desplegamos el menú de **Archivo** y hacemos clic en **Guardar** (o si quieres ir más rápido, usa el atajo Ctrl + S).

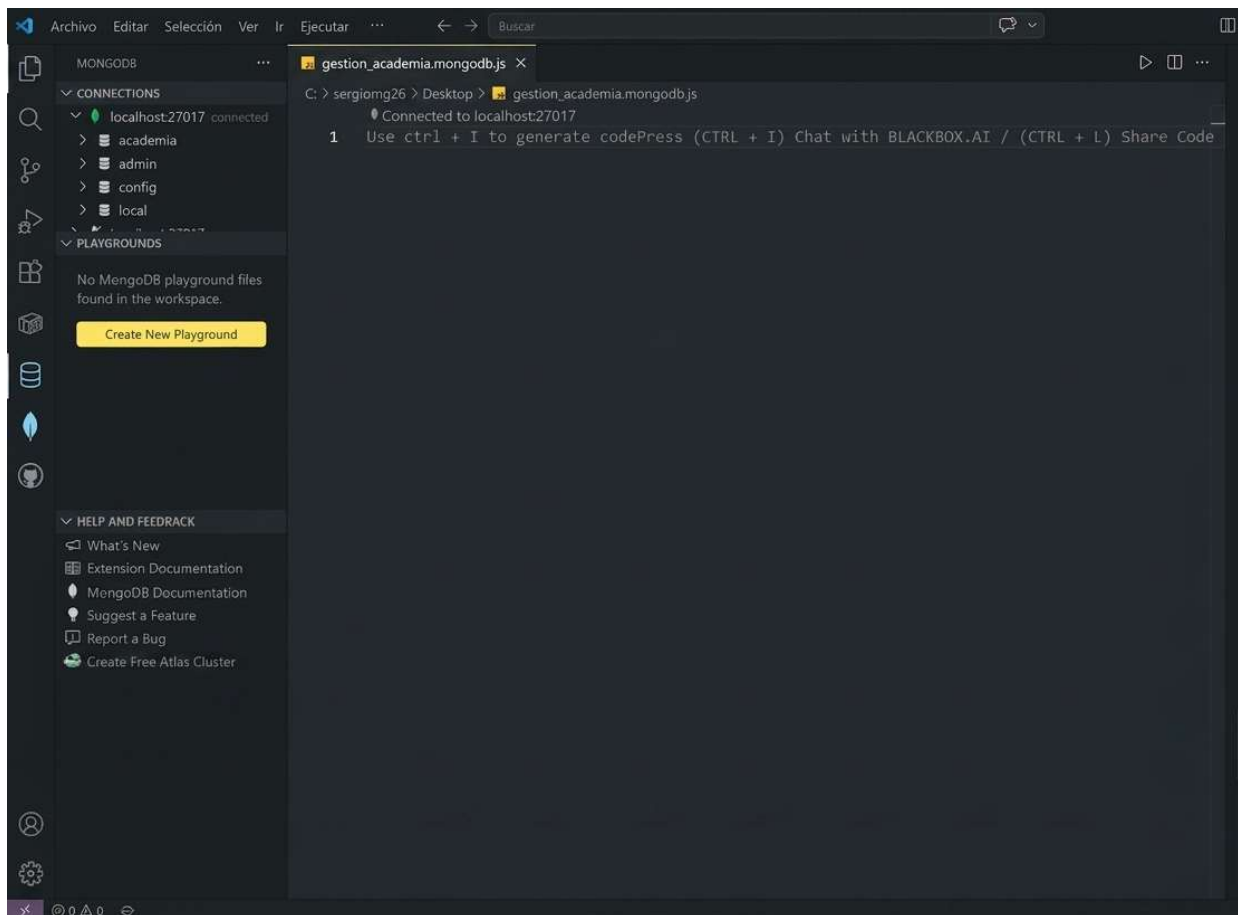


Para que el sistema reconozca el archivo correctamente, debemos guardarlo bajo el nombre **gestion_academia.mongodb.js**. Al hacerlo así, verás que el propio editor lo detecta como un script de Mongo, lo que nos facilitará la vida a la hora de ejecutar los comandos más adelante.

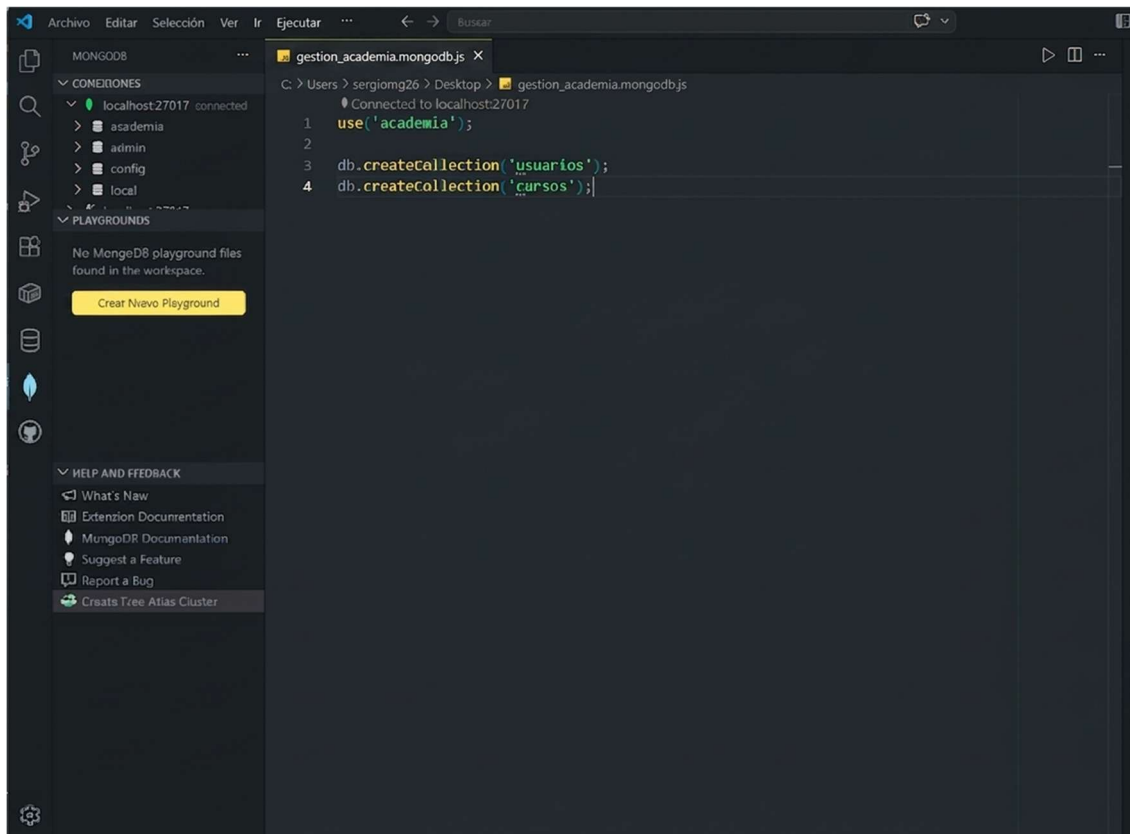


gestion_academia.mongodb.js

Una vez guardado, así es como se tiene que ver tu espacio de trabajo. Verás que el archivo ya tiene nombre y que la estructura del código está limpia. Si lo tienes exactamente así, ya estás preparado para el siguiente paso, que es lanzar los comandos al servidor.



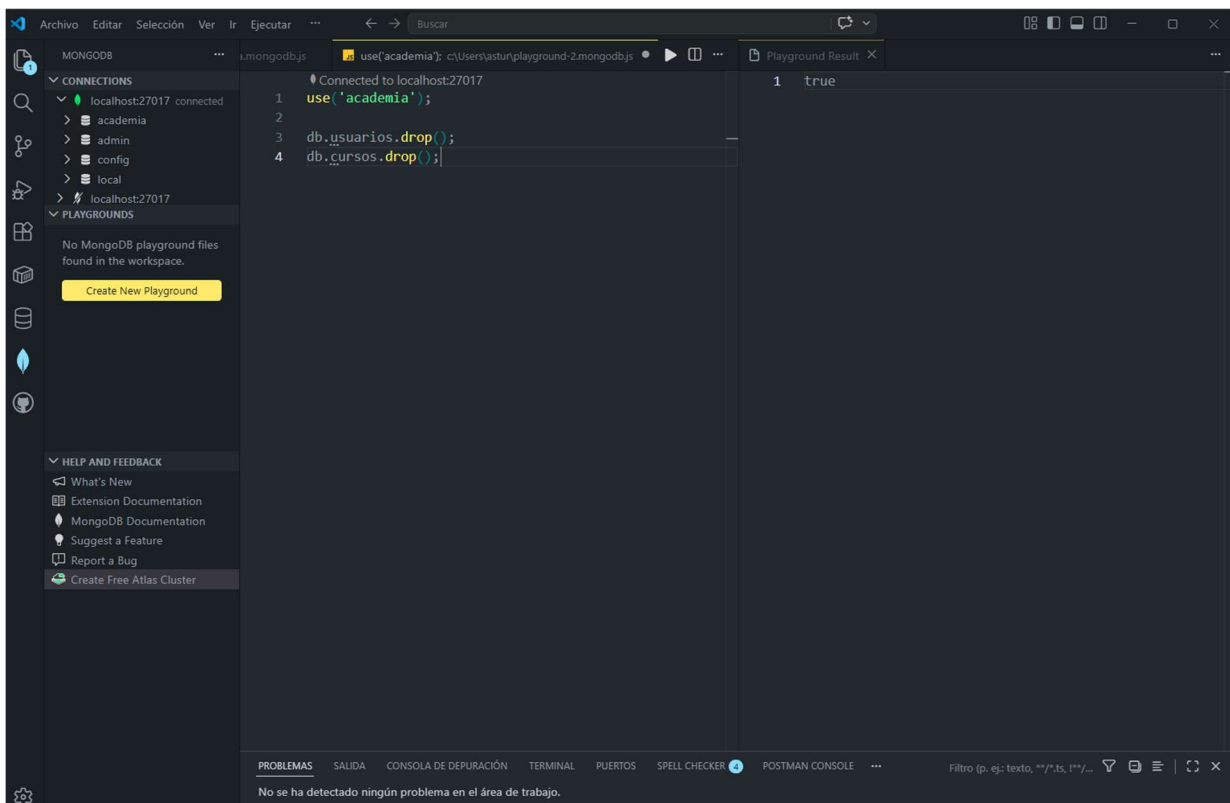
Esto es la señal definitiva de que el **programa ha reconocido el formato .mongodb.js**. Al hacerlo, Visual Studio Code activa automáticamente todas las herramientas de ayuda y resaltado de sintaxis, lo que te garantiza que el archivo está listo para comunicarse con tu base de datos sin problemas de compatibilidad.



Esta captura significa que la ha detectado como un archivo en formato .mongodb.js

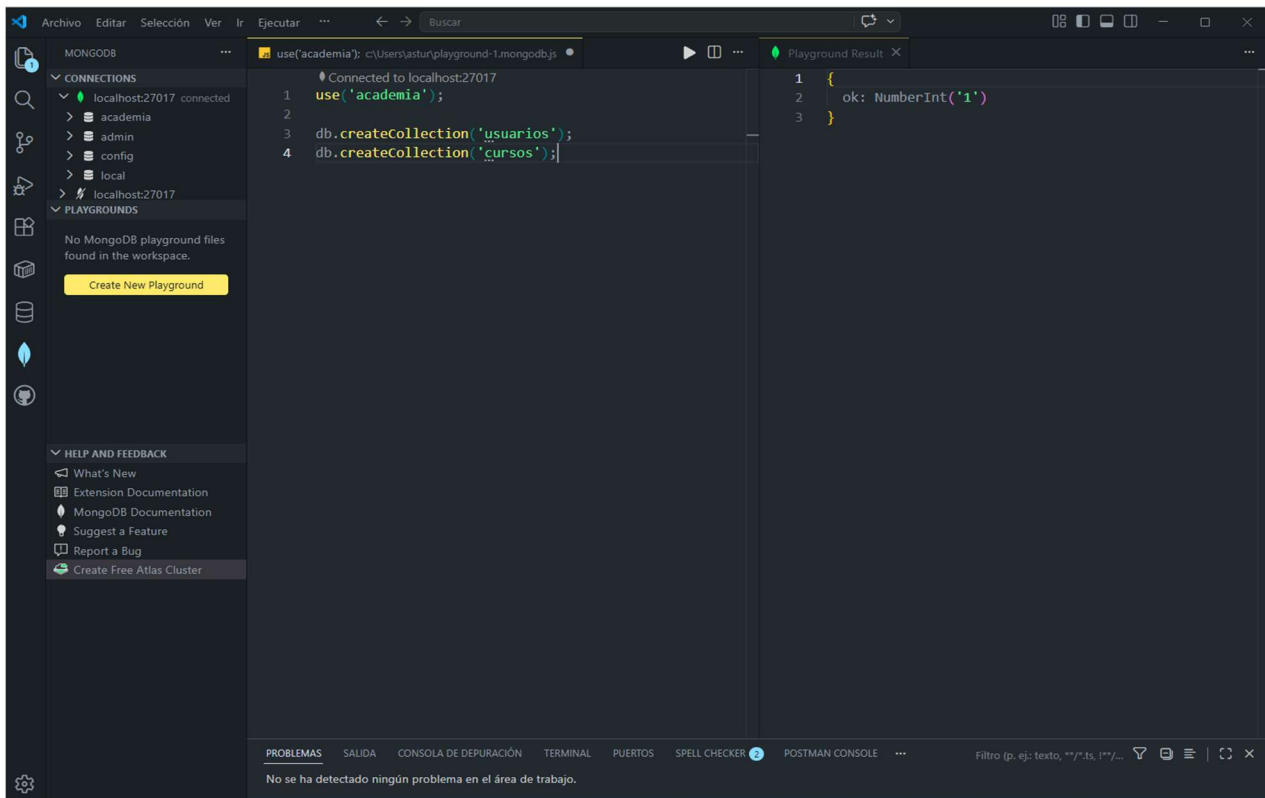
Bloque 0 – Limpieza de la base de datos academia (drop de colecciones).

Estamos ejecutando los comandos uno a uno y comprobando los resultados en la base de datos academia.



Bloque 1 – Creación de las colecciones usuarios y cursos en academia.

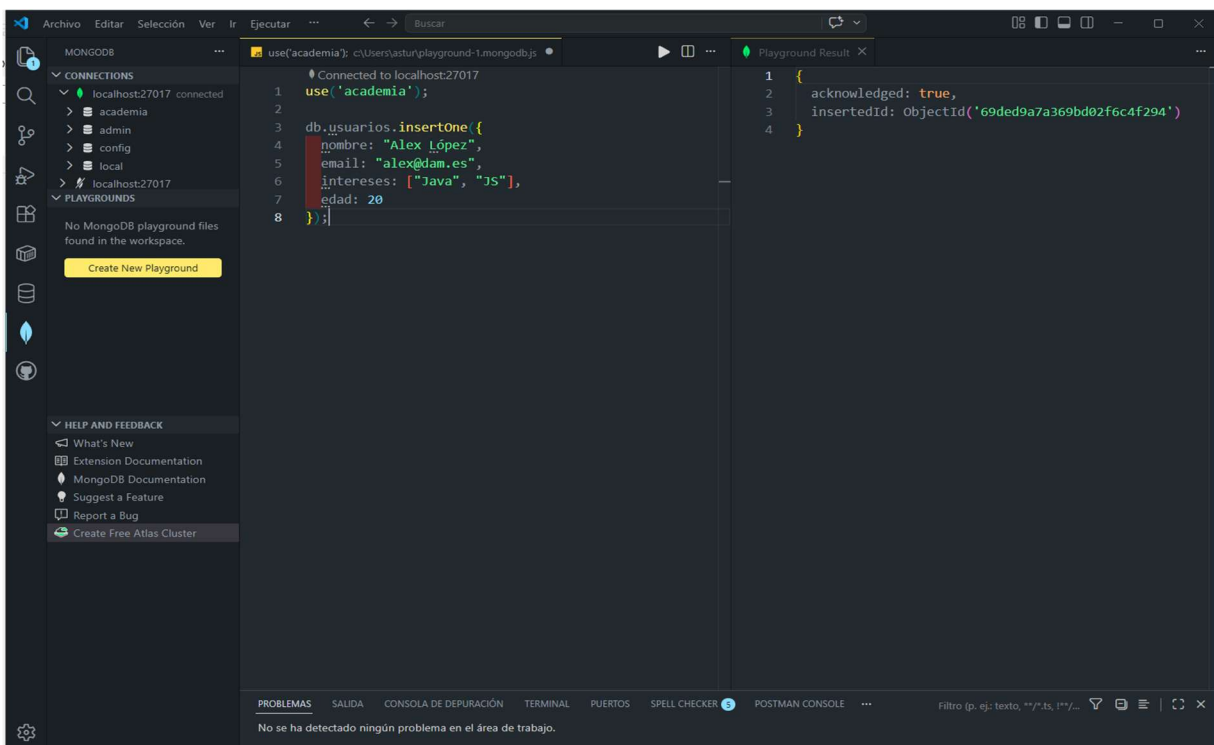
Inserción de un usuario en la colección usuarios:



En este paso seleccionamos la base de datos academia y ejecutamos los comandos `createCollection('usuarios')` y `createCollection('cursos')`. El resultado `ok: 1` indica que MongoDB ha creado correctamente ambas colecciones.

Bloque 2 – Inserción de un único usuario en la colección usuarios.

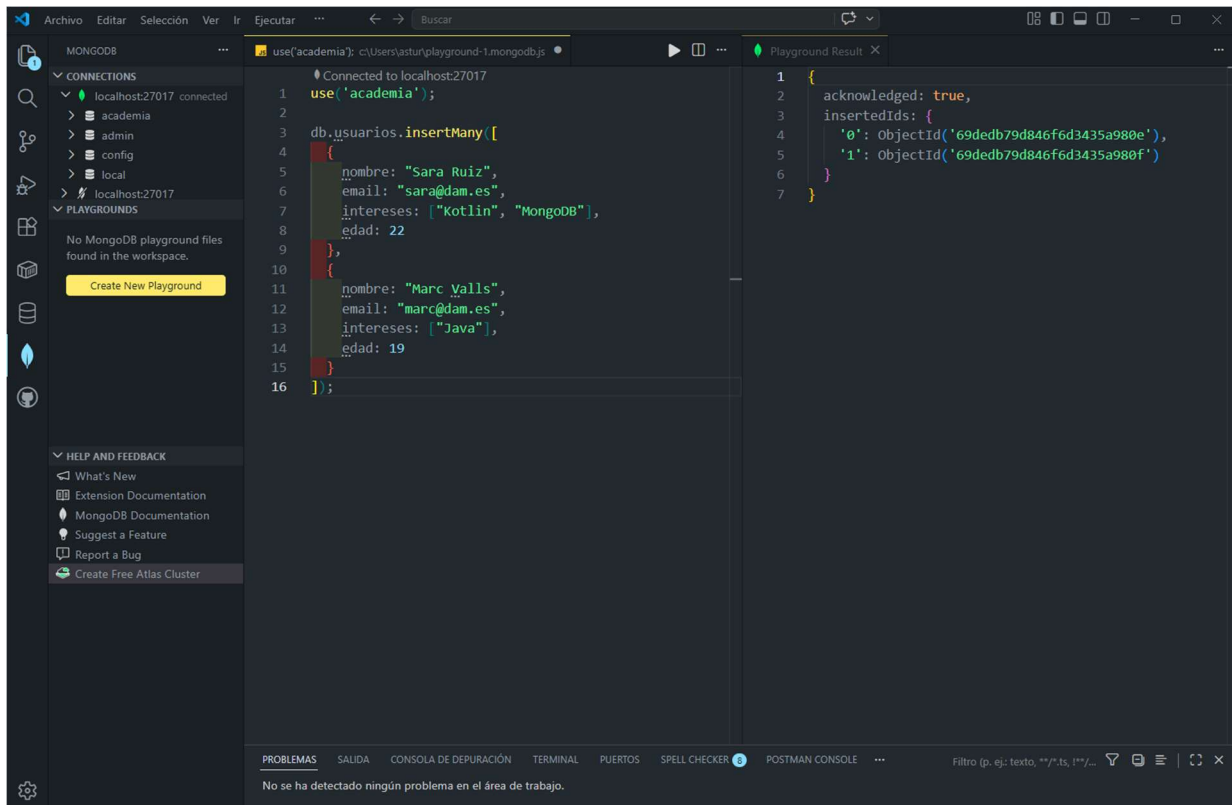
Inserción del usuario “Alex López” en la colección usuarios.



En este paso ejecutamos el comando `insertOne()` sobre la colección `usuarios` para añadir el usuario “Alex López” con su correo, sus intereses y su edad. El resultado del Playground muestra `acknowledged: true` y un `insertedId`, lo que confirma que MongoDB ha insertado correctamente el documento en la base de datos `academia`.

Bloque 3 – Inserción de varios usuarios con `insertMany()` en `usuarios`.

Inserción de varios usuarios en la colección `usuarios` con `insertMany()`



The screenshot shows the MongoDB Playground interface. On the left, the 'CONNECTIONS' panel shows 'localhost:27017' connected to the 'academia' database. The 'PLAYGROUNDS' panel shows 'No MongoDB playground files found in the workspace.' and a 'Create New Playground' button. The main editor shows the following JavaScript code:

```
1 use('academia');
2
3 db.usuarios.insertMany([
4   {
5     nombre: "Sara Ruiz",
6     email: "sara@dam.es",
7     intereses: ["Kotlin", "MongoDB"],
8     edad: 22
9   },
10  {
11    nombre: "Marc Valls",
12    email: "marc@dam.es",
13    intereses: ["Java"],
14    edad: 19
15  }
16 ]);
```

On the right, the 'Playground Result' panel shows the following JSON response:

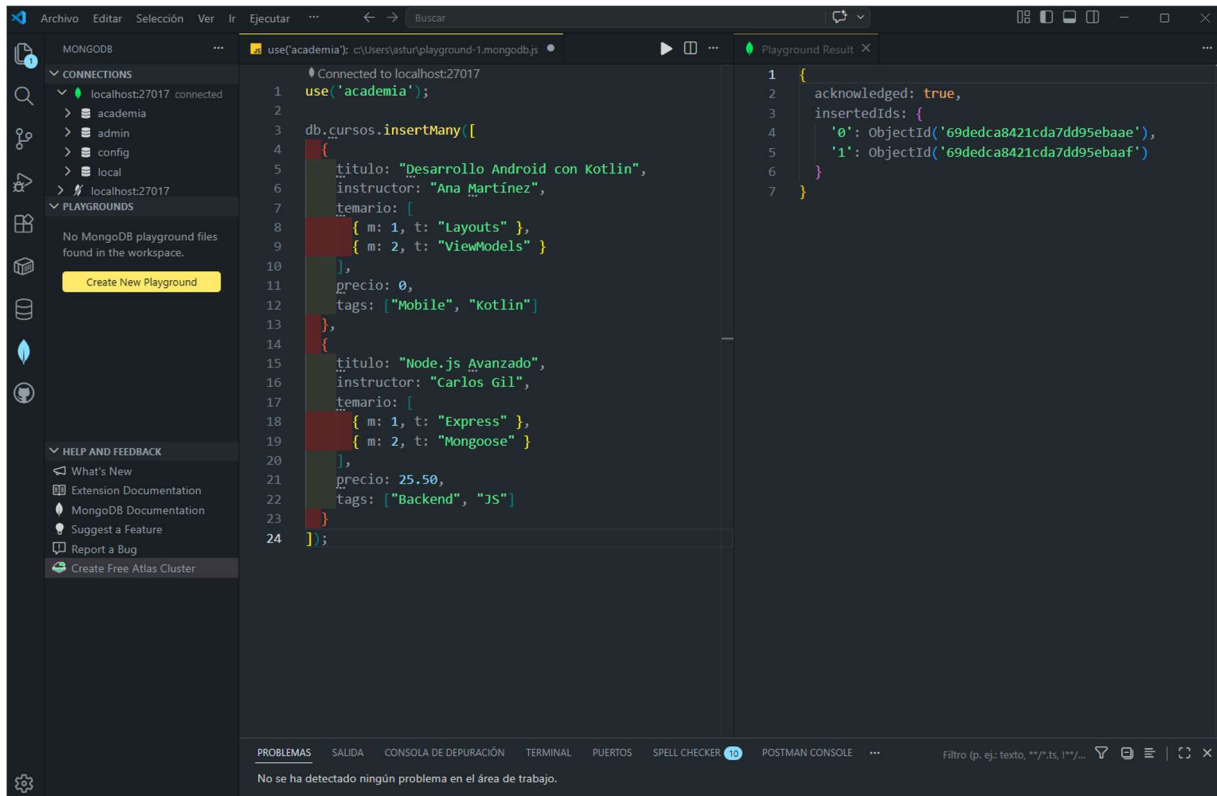
```
1 {
2   acknowledged: true,
3   insertedIds: {
4     '0': ObjectId('69dedb79d846fd3435a988e'),
5     '1': ObjectId('69dedb79d846fd3435a988f')
6   }
7 }
```

At the bottom, the 'PROBLEMAS' panel shows 'No se ha detectado ningún problema en el área de trabajo.'

En este paso utilizo el método `insertMany()` para añadir dos usuarios nuevos a la colección `usuarios`: Sara Ruiz y Marc Valls, cada uno con su correo, intereses y edad. El resultado muestra `acknowledged: true` y dos `insertedIds`, lo que confirma que MongoDB ha guardado correctamente ambos documentos en la base de datos `academia`, cumpliendo el apartado del enunciado que pide insertar varios usuarios a la vez.

Bloque 4 – Inserción de los cursos en la colección cursos.

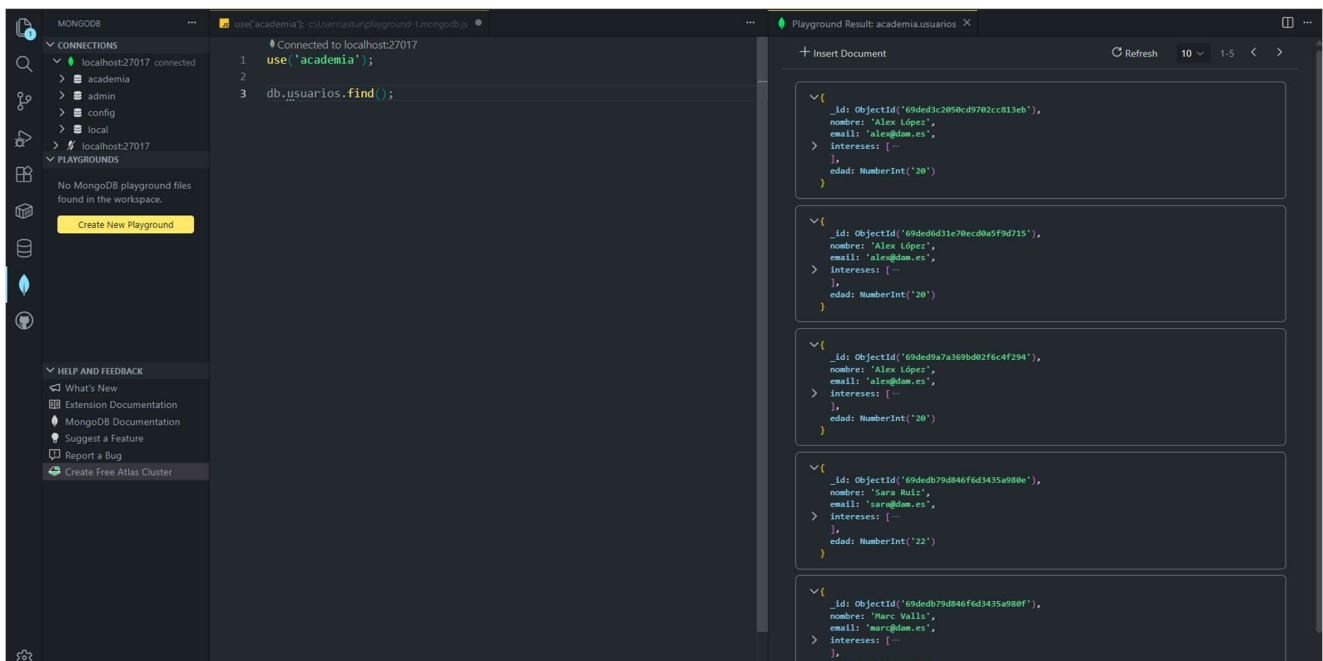
Inserción de los cursos en la colección cursos con insertMany()



En este paso utilizamos el método `insertMany()` sobre la colección `cursos` para añadir dos cursos: “Desarrollo Android con Kotlin” y “Node.js Avanzado”. Cada documento incluye título, instructor, temario (como un array de objetos), precio y etiquetas. El resultado muestra `acknowledged: true` y dos `insertedIds`, lo que confirma que MongoDB ha guardado correctamente ambos cursos en la base de datos `academia`.

Bloque 5 – Consulta de todos los usuarios de la colección usuarios.

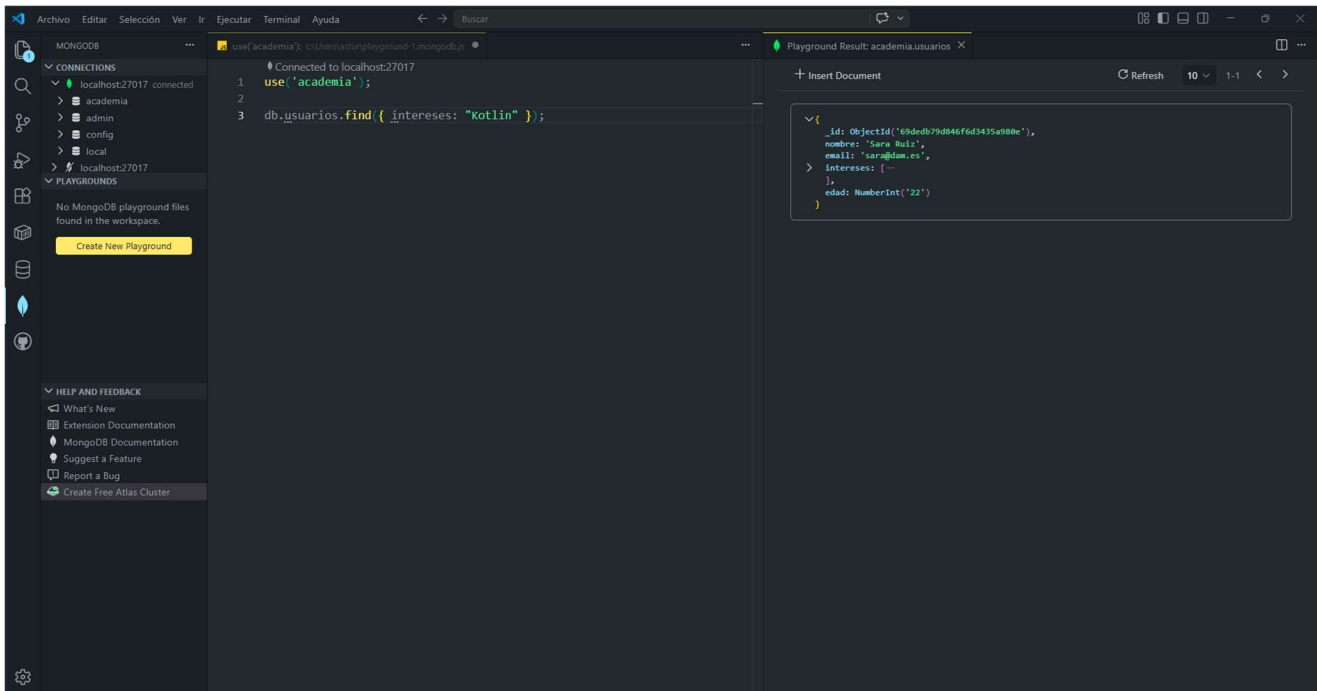
Consulta de todos los usuarios almacenados en la colección usuarios.



En este paso ejecutamos la instrucción `db.usuarios.find()` sobre la base de datos `academia` para mostrar todos los documentos de la colección `usuarios`. En el panel de resultados se ven los usuarios insertados anteriormente (Alex López, Sara Ruiz y Marc Valls), lo que me permite comprobar que las operaciones de inserción se han realizado correctamente.

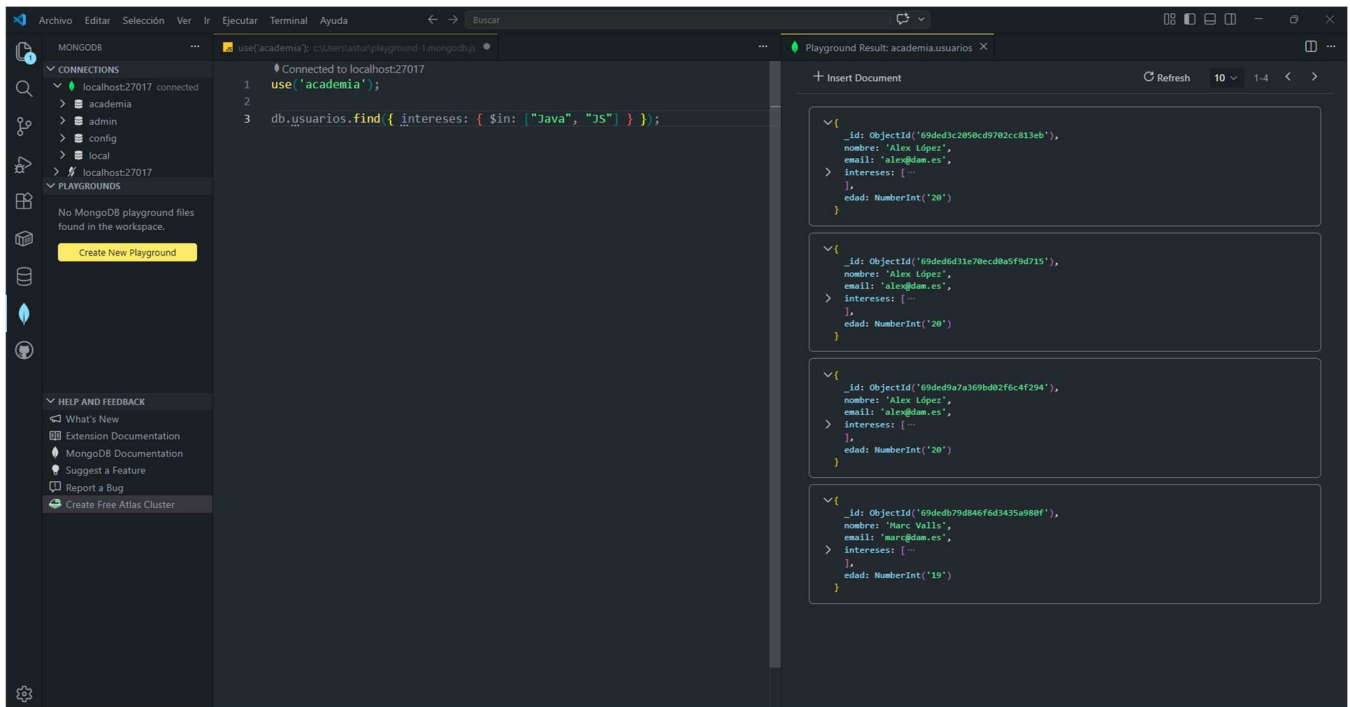
Bloque 6 – Consulta de usuarios con interés en Kotlin.

Consulta de usuarios cuyo interés incluye Kotlin



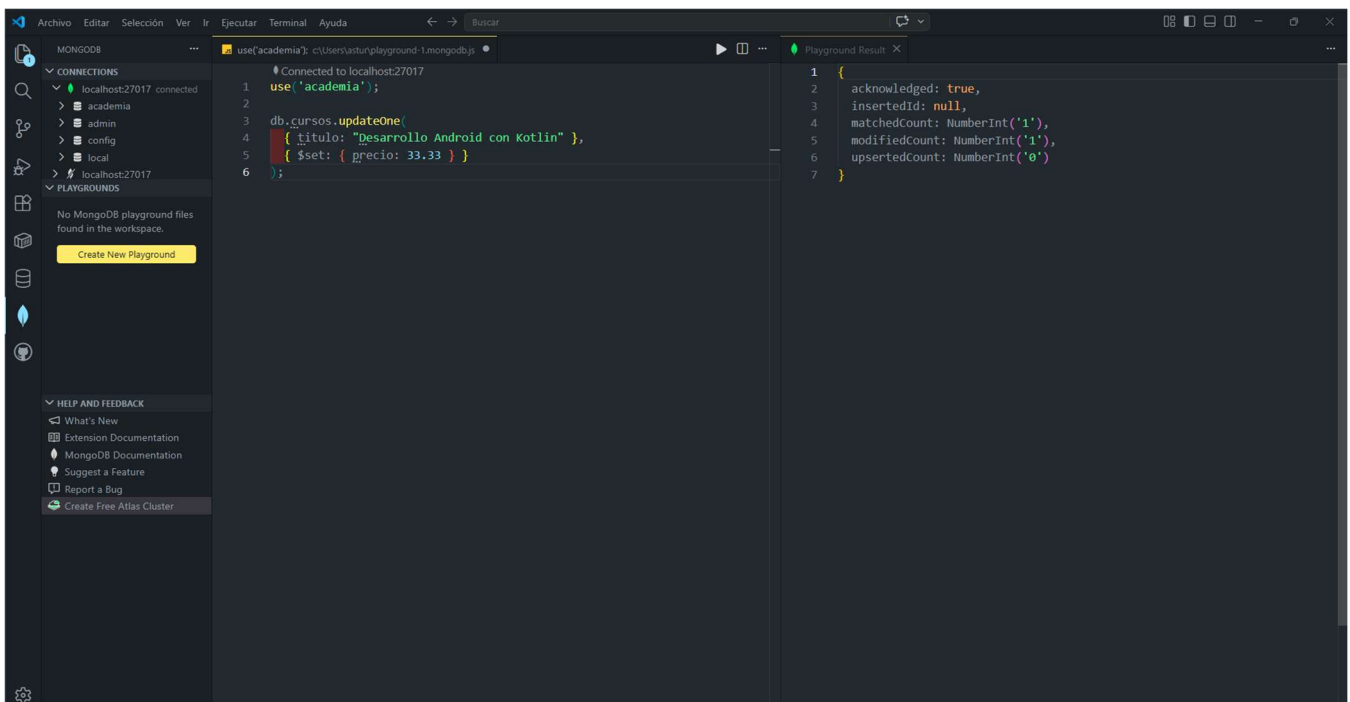
Aquí ejecutamos la consulta `db.usuarios.find({ intereses: "Kotlin" })` sobre la base de datos `academia`. MongoDB comprueba el array `intereses` de cada usuario y devuelve únicamente el documento de Sara Ruiz, que es la usuaria que tiene Kotlin como uno de sus intereses. Esta captura demuestra que la consulta de filtrado por interés Kotlin funciona correctamente.

Bloque 7 - Consulta de usuarios con interés en Java o JS usando \$in.



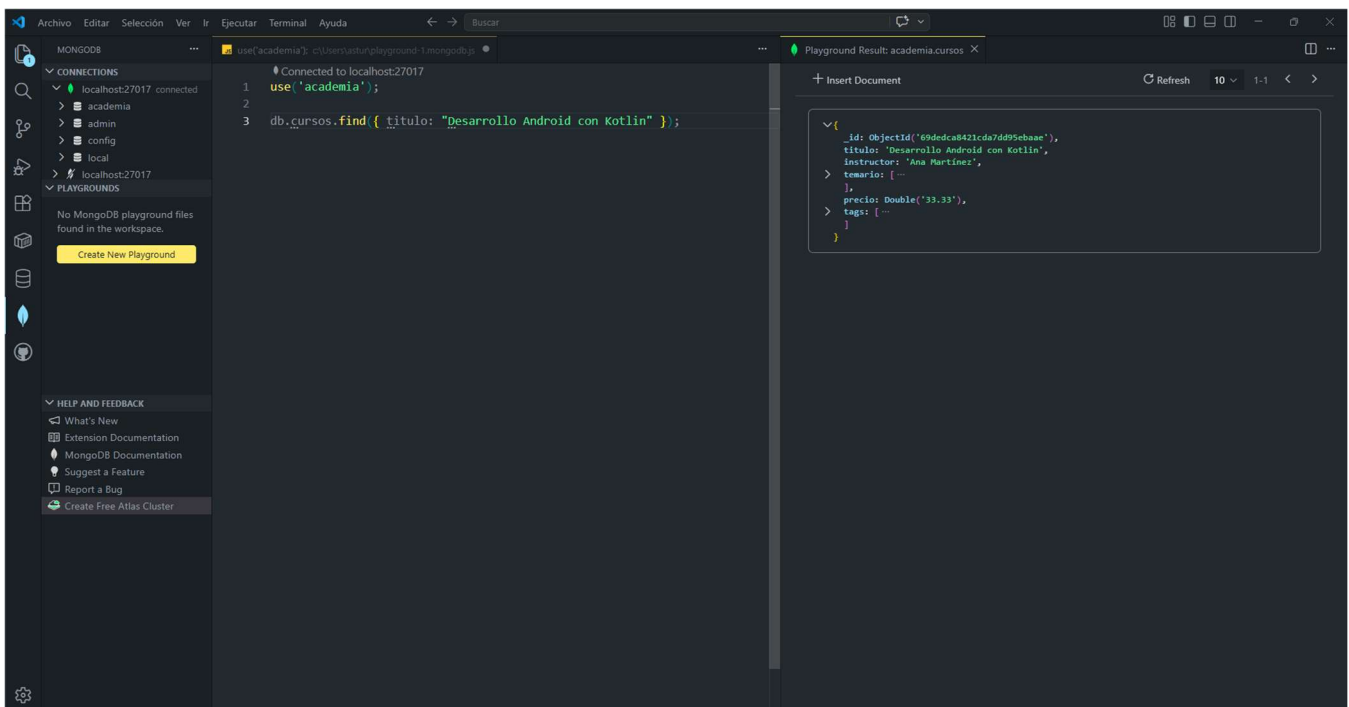
Ahora ejecutamos la consulta `db.usuarios.find({ intereses: { $in: ["Java", "JS"] } })` sobre la base de datos academia. MongoDB busca en el array intereses de cada usuario y devuelve aquellos que contienen al menos uno de los valores indicados (Java o JS). En el resultado aparecen los usuarios que cumplen esta condición, lo que demuestra que el filtro con `$in` funciona correctamente.

Bloque 8 - Actualización del precio del curso de Kotlin con updateOne() Actualización del precio del curso “Desarrollo Android con Kotlin” mediante updateOne()



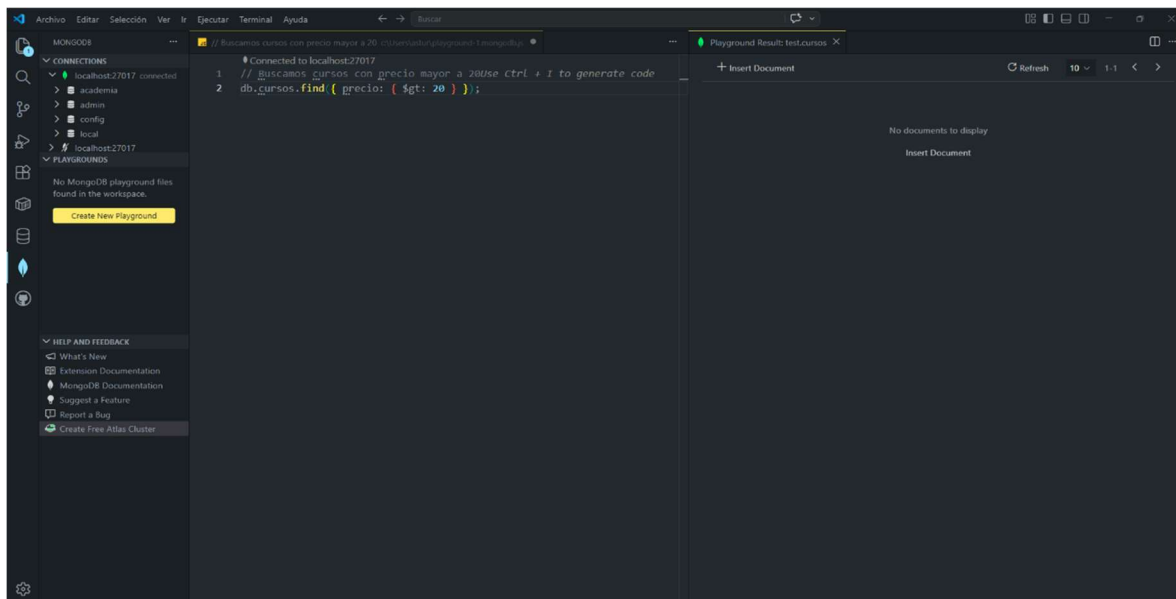
Ahora en esta captura se ve que ejecutamos la instrucción `db.cursos.updateOne({ titulo: "Desarrollo Android con Kotlin" }, { $set: { precio: 33.33 } })` sobre la base de datos `academia`. El resultado muestra `acknowledged: true`, `matchedCount: 1` y `modifiedCount: 1`, lo que indica que MongoDB ha encontrado un curso con ese título y ha actualizado correctamente el campo `precio` de 0 a 33.33, en donde modificamos el precio del curso de Android.

Bloque 9 - Comprobación del precio actualizado del curso de Kotlin. Verificación del nuevo precio del curso “Desarrollo Android con Kotlin”



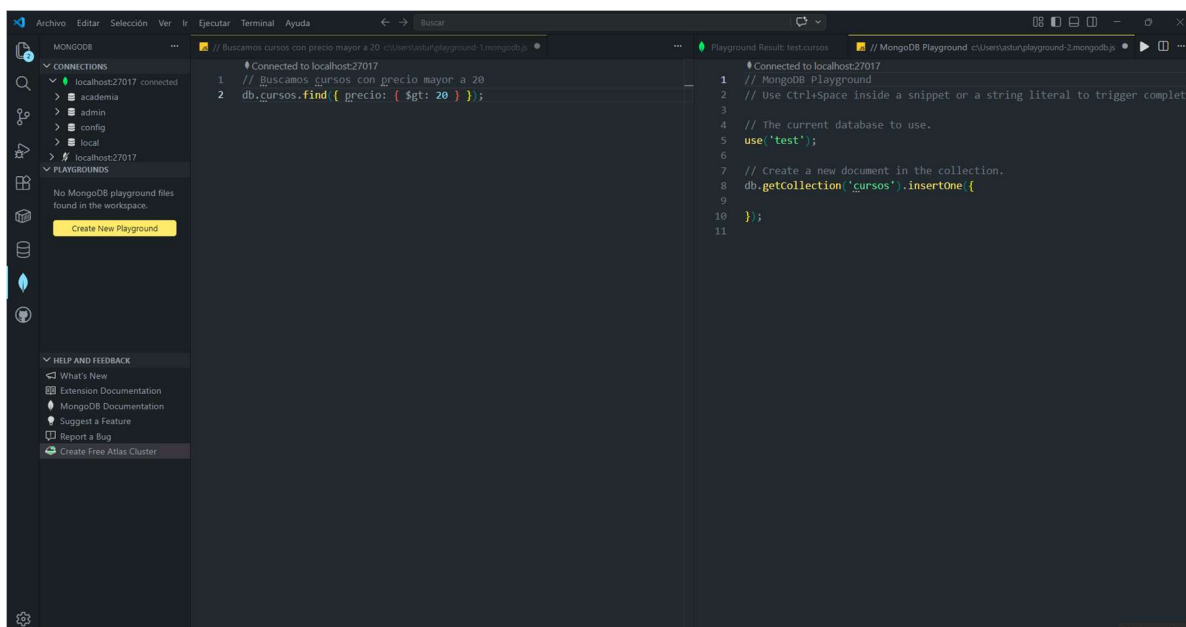
En este último código ejecutamos la consulta `db.cursos.find({ titulo: "Desarrollo Android con Kotlin" })` sobre la base de datos `academia` para comprobar el resultado de la operación de actualización. En el documento devuelto se observa que el campo `precio` aparece con el valor 33.33, lo que confirma que el cambio de precio solicitado en esta tarea o código se han aplicado correctamente en la colección `cursos`.

Consulta adicional: búsqueda de cursos con precio superior a 20 euros.



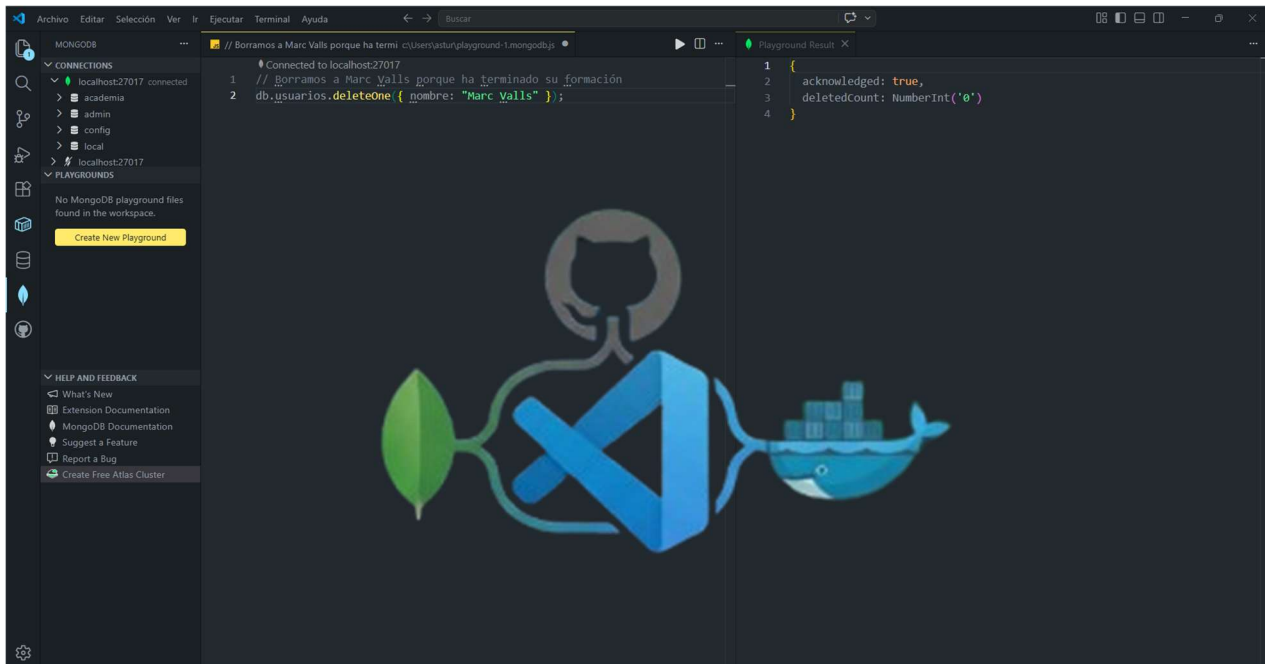
En esta consulta adicional ejecuto `db.cursos.find({ precio: { $gt: 20 } })` sobre la base de datos `academia` para buscar cursos cuyo precio sea mayor que 20 euros. En este caso el resultado aparece vacío porque en la colección `cursos` no hay ningún documento que cumpla esa condición en el momento de la consulta. Esta captura la incluyo como ejemplo extra de uso de operadores de comparación en MongoDB.

Consulta de cursos: por precio usando el operador `$gt` (mayor que)



En este ejemplo adicional preparo un Playground donde utilizo la instrucción `db.cursos.find({ precio: { $gt: 20 } })` para buscar cursos con un precio superior a 20 euros en la colección `cursos`. A la derecha se muestra el código de ejemplo generado por el propio Playground sobre otra base de datos de prueba (`test`), que ilustra cómo crear una colección y realizar inserciones desde Visual Studio Code. Incluyo esta captura como material complementario para practicar consultas con operadores de comparación en MongoDB.

Esta captura muestra **otra operación extra**, en este caso un borrado. Eliminación de un usuario de la colección usuarios con deleteOne()



En esta operación adicional ejecutamos `db.usuarios.deleteOne({ nombre: "Marc Valls" })` sobre la base de datos academia para eliminar al usuario “Marc Valls” de la colección usuarios. El resultado devuelve `acknowledged: true` y `deletedCount: 0`, lo que significa que en este momento no se ha borrado ningún documento porque no se encontró ningún usuario con ese nombre (probablemente ya había sido eliminado antes o no existía). Incluyo esta captura como ejemplo complementario de uso del método `deleteOne()` en MongoDB.

Llegados a este punto, puede parecer que solo hemos estado escribiendo comandos en una pantalla negra y moviendo piezas en Docker. Pero la realidad es otra: acabamos de orquestar la infraestructura que sostiene a las aplicaciones modernas.

Hoy ha sido una base de datos de una 'academia' en nuestro ordenador del Instituto, pero la lógica es exactamente la misma que usa Netflix para recomendarte series o Uber para localizar un coche. Hemos aprendido a levantar un servidor desde cero, a conectar herramientas profesionales como VS Code y a hablarle de tú a tú a los datos. > Este manual no termina aquí. Es solo el punto de partida para que dejemos de ser un usuario de tecnología y empecemos a ser quienes lo construyen. El contenedor está listo, la conexión es estable... ahora nos toca a nosotros a decidir qué vamos a crear o construir la siguiente vez."

